



THE UNIVERSITY *of* EDINBURGH

Edinburgh Research Explorer

High-throughput permissionless blockchain consensus under realistic network assumptions

Citation for published version:

Panagiotakos, G, Fitzi, M, Coretti, S, Russell, A & Kiayias, A 2025, High-throughput permissionless blockchain consensus under realistic network assumptions. in YT Kalai & SF Kamara (eds), *Proceedings of the 45th International Cryptology Conference*. Lecture Notes in Computer Science, Springer, pp. 69-100, The 45th International Cryptology Conference, Santa Barbara, California, United States, 17/08/25. <https://doi.org/10.1007/978-3-032-01878-6>

Digital Object Identifier (DOI):

[10.1007/978-3-032-01878-6](https://doi.org/10.1007/978-3-032-01878-6)

Link:

[Link to publication record in Edinburgh Research Explorer](#)

Document Version:

Early version, also known as pre-print

Published In:

Proceedings of the 45th International Cryptology Conference

General rights

Copyright for the publications made accessible via the Edinburgh Research Explorer is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy

The University of Edinburgh has made every reasonable effort to ensure that Edinburgh Research Explorer content complies with UK legislation. If you believe that the public display of this file breaches copyright please contact openaccess@ed.ac.uk providing details, and we will remove access to the work immediately and investigate your claim.



High-Throughput Permissionless Blockchain Consensus under Realistic Network Assumptions

Abstract. Throughput, i.e., the amount of payload data processed per unit of time, is a crucial measure of scalability for blockchain consensus mechanisms. This paper revisits the design of secure, high-throughput proof-of-stake (PoS) protocols in the *permissionless* setting. Existing high-throughput protocols are either analyzed using overly simplified network models or are designed for permissioned settings, with the task of adapting them to a permissionless environment while maintaining both scalability and adaptive security (which is essential in permissionless environments) remaining an open question.

Two particular challenges arise when designing high-throughput protocols in a permissionless setting: *message bursts*, where the adversary simultaneously releases a large volume of withheld protocol messages, and—in the PoS setting—*message equivocations*, where the adversary diffuses arbitrarily many versions of a protocol message. It is essential for the security of the ultimately deployed protocol that these issues be captured by the network model.

Therefore, this work first introduces a new, realistic network model based on the operation of real-world gossip networks—the standard means of diffusion in permissionless systems, which may involve many thousands of nodes. The model specifically addresses challenges such as message bursts and PoS equivocations and is also of independent interest.

The second and main contribution of this paper is Leios, a blockchain protocol that transforms any underlying low-throughput base protocol into a blockchain achieving a throughput corresponding to a $(1 - \delta)$ -fraction of the network capacity—while affecting latency only by a related constant. In particular, if the underlying protocol has constant expected settlement time, this property is retained under the Leios overlay. Combining Leios with any permissionless protocol yields the first near-optimal throughput permissionless “layer-1” blockchain protocol proven secure under realistic network assumptions.

1 Introduction

1.1 Overview

Permissionless blockchain protocols, beginning with Bitcoin [47], allow a large, continuously evolving set of mutually distrusting parties to agree on an ever-growing ledger of transactions as long as the majority of a certain underlying resource—e.g., computation, memory, stake—is controlled by the honest parties.

These protocols are generally leader-based, i.e., new leaders are repeatedly elected to propose new transaction-carrying blocks to be added to the ledger. Leader election is implemented by a resource-based lottery wherein the probability to win is proportional to a party’s relative portion of the overall resources dedicated to the system.

Nakamoto vs. BFT style protocols. Towards reaching agreement on the ledger, new blocks must be confirmed—which is achieved either by having (a) future leaders build on them and thereby voting for them implicitly or (b) a committee (also elected via the resource-based lottery) voting for them explicitly. Protocols that follow (a) are generally known as “Nakamoto-style” protocols (cf. [47]), in reference to the author of the original Bitcoin paper, whereas protocols that take approach (b) are referred to as “BFT-style” protocols (e.g., [32]), acknowledging the fact that they employ techniques from Byzantine fault-tolerant computing, a research area that predates blockchain systems by several decades.

Nakamoto-based protocols generally tolerate attackers controlling less than half of the network’s resources, be it work or stake, (cf. [47,29,39]) and an uncertain or even fluctuating number of participants, (cf. [30,53,6]) while permissionless BFT-based protocols typically (even though not exclusively) tolerate attackers controlling less than one-third of the resources and operate under predictable participation (e.g., [32]). In terms of settlement, BFT-based protocols offer fast finality, where confirmed blocks are permanently part of the ledger, whereas Nakamoto-based protocols offer a weaker notion of settlement, where the probability that a block remains stable slowly increases over time—at the benefit, however, that they can “self-heal” after short periods of adversarial majority [2,5]. Furthermore, handling uncertain participation as in Nakamoto-style protocols requires a network with bounded message delays (cf. [52]), whereas many BFT-based protocols exploit a level of certainty in participation to function in a partially synchronous network, where delay guarantees only apply after an initial period of uncertain duration (or where delays are limited by an unknown bound), or even in a fully asynchronous network.

The permissionless setting. In a permissionless setting, typically, many thousands of nodes are expected, something that necessitates low communication complexity: Nakamoto consensus achieves optimal communication in the sense that its amortized complexity is just a single message diffusion per block of transactions, which occur at a low rate over time; BFT-based consensus protocols require voting for each block, and thus to reduce the high message overhead, it becomes essential to elect small committees for each voting round. Consequently, in both cases, only a small subset of participants is responsible for the protocol at any given moment. This underscores the vulnerability to low-resource *adaptive* attacks, where an attacker only needs to corrupt a small group of leaders or committee members (representing a minor portion of the total underlying resources) to compromise the protocol.

It follows that adaptive security is crucial for permissionless protocols. Nakamoto consensus is adaptively secure by design since the role of the block leader is limited to creating and diffusing a block, making it “maximally ephemeral.” BFT-based protocols (that scale) only achieve adaptive security if the committees rotate frequently, ideally after every message sent (as in, e.g., Algorand [32]).

Challenges for high throughput protocol design. Throughput, i.e., the number of payload data processed per unit of time, is one of the most crucial scaling metrics for blockchains. The goal is to confirm payloads at a rate close to what the underlying network allows. In order to design *adaptively secure* high-throughput permissionless blockchain protocols, several challenges must be met:

- *Consensus is in the way:* In “conventional” blockchain protocols, block production and certification are somewhat sequential: Nakamoto protocols crucially rely on (honest) leaders building on each other’s blocks in order to achieve consensus; in many BFT protocols, one or multiple leaders propose new blocks and parties subsequently agree on which block to finalize, which evidently requires proposed blocks to reach sufficiently many honest participants. In both cases, the lag incurred by block propagation for the sake of achieving consensus inevitably sacrifices throughput since the overall network remains underutilized waiting for a single or a few blocks to propagate to all nodes. It is therefore necessary to somehow decouple diffusing payloads and consensus protocol logic in order to achieve high throughput.
- *Message bursts:* Any blockchain protocol must be able to handle the simultaneous release of a large number of *valid* protocol messages (e.g., withheld old blocks) by adversarial parties. This is particularly challenging in the permissionless setting, where nodes are usually connected by gossip networks (NB: complete point-to-point connectivity is impractical given the large number of participants). In such networks, it is infeasible to throttle messages according to their sources (a common DoS mitigation), and hence, message relaying rules are limited to coarse criteria, such as a resource lottery win and a network-wide freshness condition (e.g., the message points to the hash value of a recent block). The obvious countermeasure to message bursts is to leave sufficient bandwidth unused so that bursts can be absorbed in the worst-case (as a manifestation of this principle one can point to Bitcoin’s 10 minute expected delay between blocks); while effective, this countermeasure is completely antithetical to achieving high throughput and as a result a different mitigation approach is needed.
- *Equivocations:* In the permissionless setting, in order to achieve low communication complexity, protocol messages must be accompanied by winning lottery tickets; absent such a mechanism, it would be easy for an attacker to flood a network with invalid messages.
In the proof-of-work (PoW) setting, where lotteries are implemented by solving cryptographic puzzles against a particular message, a winning ticket is tied to that message, and hence, the only possible reuse of a PoW success

is a straightforward replay of a previous protocol message. Message replays can be easily contained in a gossiping network (e.g., by keeping a database of hashes for at least as long as a PoW remains fresh).

In a proof-of-stake (PoS) setting, however, where winning tickets are determined cryptographically, e.g., based on the output of a verifiable random function (VRFs), on a purely identity- and time-based input, there is no such tie between a specific lottery win and a message due to the fact that rewinding the lottery winning message computation is (essentially) without cost. Thus, an attacker may create so-called *equivocations*, i.e., *arbitrarily* many legitimate-looking protocol messages backed by a single lottery ticket. Therefore, such equivocations must be dealt with suitably.

1.2 Our Contributions

We present a new blockchain protocol, *Leios*, and analyze it within a new formal model that is suitable for expressing all the salient requirements of high-throughput operation in the permissionless setting. Our results in more detail are as follows.

QUEQ: A more realistic network model. The most popular network model for analyzing consensus protocols is the so-called Δ -*delay model* (and variations thereof, cf. [29,50]). This model makes the assumption that all messages sent by honest parties are delivered within a fixed time bound of Δ . As a result, the model abstracts away any details regarding how messages are propagated and any potential network congestion. Importantly, message delays are assumed *independent* of the protocol execution. It is evident that in the context of high-throughput protocols operating near network capacity, this model is too simplistic and fundamentally unrealistic.

To address this, we introduce a network model that is based on how real-world gossip protocols operate that incorporates the way that Queueing delays and Equivocations affect message transmission (hence the model name QUEQ). In more detail, any diffused message is split into a fixed size *small* header and a possibly much *larger* body, with headers being delivered predictably within Δ_{hdr} . Message bodies on the other hand are downloaded separately with their availability being subject to *queueing delays* that take place when multiple message bodies compete for network diffusion. Specifically, a particular message body will incur a delay that is a function of its size, the number of other messages that are in transit, and the *relaying policy* implemented by the network; a relaying policy specifies how relay nodes decide which message body to download and forward when multiple messages are available and compete for diffusion.

To reflect the setting of resource-based permissionless message diffusion each message header has an associated “lottery ID”; honest parties only use each lottery ID once, while malicious parties may use the same lottery ID for arbitrarily many messages (aka equivocations). The QUEQ model explicitly incorporates a simple DoS mitigation: each node locally has a preferred header—the one received first—and will never forward more than two headers per lottery ID, with

such a pair constituting a *proof of equivocation (PoE)*, i.e., proof that certain lottery ID was equivocated. Furthermore, nodes will only download the body corresponding to their preferred header. While this strategy protects against bandwidth loss (since only one message body is downloaded per lottery ID) it may leave the honest parties split in terms of the message bodies they downloaded. The QUEQ networking model leaves the resolution of this problem to the consensus protocol by offering the additional guarantee that PoEs, if they exist, are guaranteed to be delivered in a timely manner network-wide. The new network model is expressed as a UC-functionality (cf. [12]), denoted by F_{diff} .

Leios: High-throughput permissionless blockchain protocol. Our second and main contribution is Leios, a high-throughput protocol that operates over any (low-throughput) “base protocol” in a black-box manner. Leios achieves a $(1 - \delta)$ -fraction of optimal throughput for any $\delta > 0$, provided the network capacity is sufficiently large. The analysis of Leios is conducted within the QUEQ model. This makes Leios the first adaptively secure, permissionless “layer-1” protocol with near-optimal throughput operating over a more realistic network model. In particular, Leios is designed to handle queuing delays due to message bursts as well as equivocations effectively.

Bandwidth-driven concurrent block generation. Leios follows the input-endorser / fruitchains approach [29,51,39], where parties participate in a stake-based lottery to produce so-called *input blocks (IBs)*. IBs are disseminated over F_{diff} , and the overall IB data rate can be set to be arbitrarily close to the network’s capacity, enabling near-optimal throughput.

Proofs of IB availability. The IBs are included in the underlying base ledger *by reference*, which serializes the payloads and creates the final ledger. However, this decoupling of payloads from consensus introduces a data availability challenge: on the one hand, the final ledger must not include references to IBs unknown to honest parties, as this could lead to “gaps” that may or may not later be filled by the adversary (by releasing the corresponding IB), resulting in either a liveness or a safety violation (depending on how one interprets the gaps). On the other hand, parties cannot simply disregard unresolved IB references in the base protocol. These references might have been created by honest parties, but due to the high-throughput operation near network capacity, with potential message bursts, it might take some time for the referenced IB to diffuse through the network due to queueing delays.

The solution is to certify the availability of IBs using stake-based voting. A certificate ensures that at least one honest party possesses the referenced IB, guaranteeing that the IB will eventually reach all honest parties. Additionally, the ledger rules of the base protocol are modified to require certified IB references as payloads. This allows parties to accept unresolved IB references in the base protocol, knowing that the certificate ensures their eventual availability.

More efficient proofs of availability. Certifying each IB individually, however, would lead to overhead that scales with the number of IBs. This overhead therefore would consume the same constant fraction of the bandwidth, irrespective of the capacity of the underlying network, which clashes with the goal of achieving a $(1 - \delta)$ -fraction of the optimal throughput for any $\delta > 0$.

To address this issue, we introduce the concept of *endorser blocks* (EBs), which are also produced through a lottery and each collect multiple IBs (by reference). Data availability certificates are generated for EBs, with an EB becoming certified when all referenced IBs are available. Importantly, the EB rate (and hence the proving rate) need only depend on the security parameter and is therefore independent of the IB rate.

The endorsement process is designed to ensure that at least one *certified* EB covers all honestly generated IBs within a given period. Consequently, the rate of IBs referenced by the underlying ledger matches the proportion of honest parties active in the network.

Freshest-first delivery. To manage message bursts—in the Leios case, e.g., the simultaneous release of many withheld IBs by an adversary—a relaying strategy that prioritizes newer IBs over older ones is used in F_{diff} .¹ This natural and easy-to-implement relaying policy prevents an adversary from delaying the delivery of honest IBs by flooding the network with older ones.

Bounding IB delivery times. The “freshest-first” relaying policy introduces some challenges. First, observe that there must be a clear limit on how far back in time EB references can point; otherwise, establishing protocol latency would seem difficult: if an EB referencing a very old IB is certified and added to the base ledger, it could take an excessive amount of time for that IB to be delivered *to all parties*, as it would be delayed by numerous newer IBs.

Given this limit, two issues remain. First, it must be guaranteed that honest IBs are picked up by honest EBs, which requires establishing bounds on the delivery times of honest IBs. Second, it must be ensured that honest EBs are voted for: If the adversary releases an IB to a party just before it produces an EB, that party would reference the IB. It is therefore crucial to ensure that the certification process only start once the global delivery of said IB can be guaranteed, as otherwise the honestly produced EB may not obtain certification.

These issues are addressed by careful analysis based on the delivery guarantees provided by F_{diff} . Ultimately, there is a tradeoff between throughput optimality and latency: in essence, the closer the IB data rate is to the network capacity, the longer one needs to wait before including IBs in EBs and before voting on EBs for certification.

Equivocations. Recall that equivocations caused by adversarial IBs can lead to a split network where honest parties prefer different IBs for the same lottery ticket. It must therefore be ensured that such equivocated IBs are not referenced by

¹Note that IBs with timestamps from the future can simply be ignored.

honest parties. Fortunately, it can be shown that, in the event of a split, F_{diff} guarantees that all honest parties receive a proof of equivocation (PoE) within a well-defined time bound—and this fact can be exploited to prevent that honest parties reference equivocated IBs.

Pipelined protocol architecture. Given the number of stages required for IBs, EBs, and their associated votes to propagate through the network and form the necessary certificates, Leios employs a pipelined architecture. This ensures that the concurrent generation of IBs continues uninterrupted, preventing bandwidth from being wasted while votes and endorsements are collected by the protocol participants.

Properties of Leios. For ease of presentation we present two variants of Leios: Simplified Leios, which is useful to understand the approach and techniques involved, and Full Leios. Simplified Leios, for a security parameter L and any constant $\delta > 0$, achieves

- a throughput arbitrarily close to $(1 - \delta)\eta\alpha_H$, where α_H denotes the fraction of honest stake and where η is the ledger quality of the base protocol,²
- latency worsened by at most $\mathcal{O}(\delta^{-3}L)$,
- security error $e^{-\Omega(L)}$, and
- fairness, meaning that the fraction of adversarial input blocks making up the final ledger is equal to the fraction of corrupted stake,

against an adaptive adversary corrupting up to less than half of the stake (assuming the base protocol is also secure against such an adversary).

Note the factor η in the throughput is undesirable, especially when the underlying ledger is a Nakamoto-style blockchain [26], as is the latency/security tradeoff.

Full Leios does not suffer from these issues and achieves, for any $\delta > 0$,

- a near-optimal throughput of $(1 - \delta)\alpha_H$ relative to network capacity,³
- latency worsened by at most $\mathcal{O}(\delta^{-3} \log \delta^{-1})$ compared to the base protocol,
- security error independent of the latency penalty, and
- fairness, as above,

against an adaptive adversary corrupting up to less than half of the stake (assuming the base protocol is also secure against such an adversary). The key insight is that one can circumvent the impact of possibly low underlying ledger quality by having pipelines cross-reference each other.

²Intuitively, a chain quality of η means that there is an honest block among any sequence of blocks that was produced during η time slots.

³In any shorter term, this level of throughput is essentially optimal since even without the burden of consensus one may at best hope to reach a fraction of α_H of network utilization, since the adversarial nodes can always “waste” the remaining bandwidth.

For convenience, both variants of the protocol are first presented assuming the adversary does not create any equivocations. We then demonstrate how we can exploit the structure of Leios and proofs of equivocation to handle equivocations and ensure they do not impact the protocol.

In our presentation we focus on the PoS setting, but we also briefly explain in Appendix H, how the construction can be extended to other resources such as PoW. Furthermore, if the base protocol supports dynamic participation (as, e.g., Nakamoto-style protocols generally do), both Simple and Full Leios remain live in the face of fluctuating participation. During times of low participation, the throughput of Leios gracefully falls back to that of the base protocol, without affecting safety or liveness. As participation improves, the high-throughput properties of Leios take effect anew. In this way, for instance, combining our scheme with Ouroboros [39,21] one can obtain an optimal throughput PoS protocol that supports dynamic participation.

1.3 Related Work

Modeling networking for blockchain protocols. Previous analyses of blockchain feature a number of increasingly more realistic network models. Early work was based on so-called “message passing rounds” [29] or the aforementioned Δ -delay model (e.g., [21]).

Bagaria *et al.* [7] refine the Δ -delay model by setting $\Delta := |m|/C + D$ to diffuse a message m , where C is the capacity of the network (in bits per second) and D is a propagation delay (at minimum equal to what is imposed by the speed of light). The protocol they analyze in this model (see below) proceeds in rounds of length Δ and, in order not to overwhelm the network, keeps the data volume to be processed per round below ΔC . The limitation of this approach is that it accounts for neither message bursts nor equivocations.⁴

Neu *et al.* [49] were the first to introduce a model that separates messages into headers and bodies. To diffuse a message, a party first uploads it into a hypothetical *cloud*. A message header diffuses with a fixed delay Δ_{hdr} , and any message body may be downloaded once its header is received, but with a rate bounded by the capacity C . While this model makes an important first step towards recognizing the issue of message bursts (as parties now must explicitly choose which message bodies they download), it still does not account for equivocations: for example, equivocations can happen on the header level, but the model assumes that all headers become available within Δ_{hdr} , irrespective of how many equivocations the attacker attempts to diffuse. Furthermore, in terms of message bursts, the immediate availability of block bodies for download (once the header has been received) does not realistically model diffusion over gossip networks, where bodies only become available after they spend some time traversing the network wherein they may encounter other messages that can delay them further, depending on the relaying policy of the network.

⁴The latter (equivocations) is not needed in [7], as the protocol is PoW-based.

Scaling Nakamoto consensus. A classic approach to overcome the consensus protocol overhead is to decouple consensus blocks from the dissemination of payload data. In the context of blockchain protocols this was highlighted first by Bitcoin-NG [25], where each consensus-block producer continues to append new ledger-transaction blocks (without computing new PoWs) until the next consensus-block gets published. However, this protocol only provides good throughput with game-theoretic guarantees, but not under Byzantine corruption, as a malicious miner can append his consensus block immediately to its previous consensus block and thus censor all transaction blocks the previous miner has appended.

The input-endorsers/fruitschains [29,51,39] technique was originally devised to achieve fairness in Nakamoto-style consensus (i.e., the fraction of blocks a party is able to contribute to the settled blockchain is proportional to their resources), which is provably impossible in pure Nakamoto consensus [26].

Bagaria *et al.* [7] insightfully observed that the input-endorsers technique can be effectively applied towards achieving high throughput by keeping the underlying Nakamoto protocol “minimal” (and safe) but pushing the input-block production towards the bandwidth limit. Their resulting *Prism* protocol, based on PoW, achieves optimal throughput in the limited network model mentioned above. Note that a similar outcome was achieved in a more involved way by Fitzi *et al.* [27] and Yu *et al.* [59] by running multiple quasi-independent blockchain protocols in parallel and merging the multiple chains into a single ledger. This approach is also known as *Parallel Chains*.

A different approach to decoupling consensus blocks from the payload data was given by Zhang *et al.* [61] where blocks pre-announce transactions to be included in later consensus blocks at a certain minimal distance—to allow for the respective transactions to spread to all mempools before effective inclusion. Ideally, this guarantees that upon retrieval of a block, all included transactions are already in the local mempool, thus minimizing the amount of data to be downloaded at the time a given block is published. Note that, without additional measures, this method cannot provide optimal throughput: it is only effective for the payloads of honest blocks while, in general, the payloads of dishonest blocks still need to be downloaded at the time the block is published (as they may have been pre-announced as required, but not pre-distributed).

Equivocations. Neu *et al.* [49] highlighted first the issue of equivocations in PoS protocols. From our perspective it is worth highlighting that all mitigations offered in [49] impact throughput (or security) as an equivocation adversary may waste honest parties’ bandwidth by leading them into pursuing protocol histories that are eventually abandoned or invalid. This problem was identified by Kiffer *et al.* [40] who proposed an equivocation defense that has parties download the bodies of at most one version of each block while achieving longest-chain consensus purely on the chain of headers. This leads to situations in which some parties have downloaded a block that ultimately remains in the longest chain while others have not. They address this via equivocation proofs, which, when included in a later block (before some deadline), result in the contents from the corresponding block being annulled (i.e., they are ignored in the ledger). While

equivocation proofs rectify PoS blockchain security, the resulting throughput offered is sub-optimal, and further, the approach impacts latency as well.

Scaling BFT consensus. Classical PBFT [13] suffers from poor throughput since, even in its steady state, a new block is proposed only once during multiple protocol rounds. Hotstuff [57] improved over PBFT by a pipelining construction to allow for a new block to be proposed during every single round. Still, throughput of Hotstuff is suboptimal since the “passive” replicas’ upload links are not utilized during each leader’s round. To remove this asymmetry, Schiper et al. [46] proposed a PBFT protocol where multiple leaders propose blocks concurrently, with respective follow-up proposals given in [56,33,3].

Another line of high-throughput protocols was initiated by Keidar *et al.* [36], where, in each round, all parties extend a block DAG, and a superimposed sequential leader-election process is applied to eventually agree on a serialization of the DAG blocks. Variants thereof were given in [19,55,37].

Interestingly, all of the above “parallelized” protocol variants can be seen as analogs of input endorsing for throughput as in [7], or vice versa: blocks are proposed concurrently, and eventually ordered by a slim consensus mechanism.

From our perspective it should be emphasized that the above protocols are designed for the *permissioned* setting and would have to be adapted suitably to work in a permissionless environment while retaining adaptive security, which appears non-trivial. A particular approach could be to prove that the protocols satisfy the notion of *player replaceability* [32] and then use sortition to deploy them in a permissionless setting. However, this would also require establishing that the protocols are able to handle message bursts and equivocations—essentially, one needs to show that they remain secure in the QUEQ model, and if they are not, determine whether they can be suitably adapted.

Other work. Further details on previous work, and how our tooling relates to it, are given in supplementary material Appendix A.

Organization of the Paper. Section 2 succinctly presents a model for capturing the security of ledger protocols, including: the protocol interface, security definitions, and the hybrid functionalities that we are going to use. The QUEQ network model is described in detail in Section 3. Sections 4 and 5 are dedicated to the description and analysis of Simple and Full Leios, respectively. Handling equivocations is explained in Section 6. Due to lack of space, we defer to the Appendix details about the security analysis and the hybrid functionalities we use.

2 Preliminaries

2.1 Security Definitions

Model. To capture the security of ledger protocols, we adopt a model providing parties access to a number of hybrid functionalities $\mathcal{F}_i$ reflecting infrastructural

assumptions such as a (low-throughput) network model, VRFs, signatures, etc. In particular, it is assumed that there is a key registration functionality that parties can use to distribute keys consistently among each other, as in [21].

This work considers only *synchronous* protocols in which parties have access to a shared clock that proceeds in *slots*. A ledger protocol Π has access to these hybrids and provides the interfaces below to the environment; for simplicity, only PoS protocols are considered here, but the definitions generalize to other protocols based on other resources.

As our overlay protocol will ultimately rely on the base protocol to settle protocol metadata rather than low-level transactions, we keep the transactions abstract and refer to them as *payloads*. In contrast, the payloads of the overlay protocol itself are conventional ledger transactions.

The interface between the environment and a ledger protocol is as follows:

- Initially, the protocol takes an input (INIT, V) , where V is a validation predicate taking a payload p as input. It replies to this by producing an output $(\text{STAKE}, \text{SD})$ containing the *stake distribution* $\text{SD} = ((P, s_P))_P$, where $s_P \in [0, 1]$ and $\sum_P s_P = 1$.
- The protocol takes input (FTCHLDG) and then returns $(\text{FTCHLDG}, \tilde{L}, L)$, where $\tilde{L}$ and L are vectors of payloads as follows: $\tilde{L}$ represents the state of the *tentative* ledger state (including a possibly unsettled part), and L represents the *settled* part of the ledger.
- At any point the environment accepts messages (FTCHPLD) from parties running the protocol and replies to these messages immediately with a message $(\text{FTCHPLD}, p)$, where p is a payload.⁵

The way FTCHPLD works models the idea that in practice a protocol would fetch transactions from a local mempool when needed (i.e., the mempool is modeled as part of the environment).

Security. The security of the ledger protocol is captured w.r.t. an attacker that may corrupt an arbitrary number of parties as long as the stake corresponding to corrupted parties P —based on the stake distribution SD output initially—remains below a certain threshold (typically $1/3$ or $1/2$).⁶ Throughout this paper, α_H denotes the fraction of honest stake.

Ledger protocols are expected to satisfy the usual two fundamental properties: Persistence and Liveness, where Persistence requires that all parties have consistent views of the ever-growing ledger and Liveness requires that honest payloads make it into the ledger.

⁵For simplicity, we restrict the environment to only input p that satisfy $V(p)$ and to input the same V to all parties.

⁶For simplicity it is tacitly assumed that the protocols considered in this work are well-formed in that they output the same stake distribution to all parties. In particular, it is assumed that protocols have a way for parties to coordinate the initial stake distribution as well as coordinate the key registration process, e.g., as done in [21].

In the following, let $L_P^{(t)}$ be the random variable corresponding to the settled ledger (output via the FTCHLDG command) as seen by party P at time t , and $\mathbf{p}_{P,\tau}$ be the vector of payloads fetched from the environment by party P at time τ via FTCHPLD.

Definition 1 (Persistence with parameter w). *For all honest parties P, P' and all times $t \leq t'$, it holds that (i) $L_P^{(t)}$ is a prefix of $L_P^{(t')}$ and (ii) $L_{P'}^{(t-w)}$ is a prefix of $L_P^{(t)}$.*

Definition 2 (Liveness with parameters r and u). *For all times t , the following holds: if some payload p is included in the response to a FTCHPLD request made by any honest party during r rounds, i.e., in each round $\tau = t, t+1, \dots, t+(r-1)$, $p \in \mathbf{p}_{P,\tau}$ for all honest P , then p will be contained in $L_P^{(t+u)}$ for each honest party P .*

The above (standard) liveness definition is satisfied by our overlay protocol. In contrast, the base protocol is assumed to satisfy a well-studied [50] stronger flavor of liveness that is coupled to ledger quality—to express that good payloads eventually get included—as follows:

Definition 3 (Ledger Quality with parameters η and u). *For all times t , the following holds: at least one honest payload vector from the η slots $\tau = t, t+1, \dots, t+(\eta-1)$ will be contained in the common prefix of $L_{P_i}^{(t+u)}$ over all honest parties P_i .*

Throughput. The throughput of a ledger protocol is defined as the average growth rate of the *sanitized* honest ledger. The sanitized honest ledger at time t , denoted $L_{\text{san}}^{(t)}$, is the common prefix of all honest settled ledgers $L_P^{(t)}$ without (a) duplicate payloads (i.e., keeping only, say, the first copy of each payload) and (b) payloads that were not input by the environment via FTCHPLD.

Moreover, the throughput of the ledger protocol is defined w.r.t. an environment that inputs (globally) unique payloads whenever requested by the protocol. This is motivated by the fact that when a (reasonably implemented) practical system is under high load, the mempools commonly contain unique transactions. Environments of this type are referred to as *unique-payload environments*.

Definition 4. *A ledger protocol has $(\varepsilon(t), R)$ -throughput if for every $\gamma > 0$, there exists a time $t_0 \in \mathbb{N}$ such that in an execution with any unique-payload environment,*

$$\Pr \left[|L_{\text{san}}^{(t)}|/t < (1 - \gamma)R \right] \leq \varepsilon(t)$$

for all $t \geq t_0$.

2.2 Verifiable Random Functions and Lotteries

A *verifiable random function (VRF)*, defined by Micali *et al.* [44], is a cryptographic primitive that allows a party P to create a key pair consisting of a

secret evaluation key and a public verification key such that: (a) the public key implicitly defines a function f (b) with the secret key, P can evaluate f at any input x , obtaining an output y and a proof π ; (c) given the public key of P , anyone is able to verify, using π , that y is indeed the output corresponding to x ; (d) with only the public verification key, outputs of the function cannot be predicted, and in fact appear random.

The above guarantees are abstracted by an idealized functionality F_{vrf} (cf. Figure 6), whose full description is provided in Appendix C. The intuitive guarantees outlined here are sufficient for understanding the main body of this paper. F_{vrf} can be realized as in [21].

VRFs can be used to implement a stake-based lottery by evaluating the VRF on a lottery identifier and requiring that the output y be below a certain (stake-dependent) threshold.

2.3 Stake-Based Voting

Leios requires a voting scheme that satisfies the following two conditions: (1) if all honest parties vote for a particular message, they are able to generate a certificate for that message; (2) if no honest party votes for a particular message, the adversary is unable to create such a certificate.

The above guarantees are abstracted by an idealized functionality $\mathcal{F}_{\text{sbv}}$ (cf. Figure 8), whose full description is provided in Appendix E, where a secure realization based on stake-based voting is provided as well. The intuitive guarantees outlined here are sufficient for understanding the main body of this paper.

3 QUEQ: A More Realistic Network Model

An important contribution of this work is QUEQ, a new, more realistic network model for analyzing high-throughput consensus for permissionless blockchains (cf. Section 1.2). The model is based on how real-world gossip protocols operate and captures the guarantees they provide.

Therefore, first an overview of gossip-protocol design is provided below, including a sketch of how one can safely implement high-throughput gossip against Byzantine adversaries. The sketched protocol serves as a justification for the QUEQ model, the most realistic model used to analyze blockchain protocols to date, accounting for QUEuing delays and so-called EQUIvocations. The QUEQ model is described as a UC functionality F_{diff} .

Real-world gossip protocols. Gossip protocols commonly use an expander-like overlay network with low degree and short diameter. The main idea is for new messages to spread by having parties forward them to their neighbors in the network. In order to avoid receiving messages multiple times from different neighbors (which wastes bandwidth), messages are split into small headers and (potentially much) larger bodies; a node, given a header, then decides from which

neighbor to download the corresponding body. Note that when aiming to maximize throughput, it is crucial that each body only be downloaded once.

Additional challenges arise in an adversarial environment: to avoid straightforward DoS attacks in which adversarial nodes simply flood the network with (useless) messages, it is important that (i) the application layer (e.g., consensus) restrict the rate of valid messages sent, and (ii) the network layer use the application layer to verify the validity of messages received and drop illicit ones. This is typically accomplished by implementing the lottery using a proof-of-X mechanism, where X can be stake, memory, work, etc.

However, even when using PoX, the adversary is still able to attack the network by buffering legitimate messages and releasing them in bulk at some point in the future. Furthermore, when using proofs of *stake*, which are adopted in this work, the adversary is able to *equivocate* messages, i.e., to send arbitrarily many different messages for the same lottery ticket. Thus, a DoS-safe network layer has to suitably manage equivocated messages.

The concrete gossip protocol after which F_{diff} is modeled works as follows: Messages are split into headers h and bodies b with the property that there is a public function $\text{match}(h, b)$ that tests whether a particular b matches a particular header h ; furthermore, it is assumed that it is infeasible to find two bodies that match the same header and that a lottery ID $\text{LID}(h)$ is inferable from the header.⁷ Lottery IDs are assumed to be from some totally ordered set and can thus also act as priorities.⁸

Header diffusion works as follows:

1. The diffusion of a message is initiated by the sender by first sending its header to all of its neighbors.
2. When a party receives a new header, it is not directly forwarded but rather output to the environment (which corresponds to the application layer).
3. When a party receives a header as input from the environment, they forward it to all of their neighbors.

The detour via the environment (in Steps (2)–(3)) “externalizes” message validity, which commonly depends on the state of the application layer. The idea is that an honest party verifies a received header and re-inputs it into the network layer if it is valid, thereby contributing to its diffusion.

In order to protect against equivocations, Step (3) is only executed if the party has not already seen multiple versions of the message. Note, however, that the first equivocation is still forwarded to the neighbors, with the goal of spreading a *proof of equivocation (PoE)* (i.e., two different messages for the same lottery ticket) throughout the network.

Body diffusion works as follows:

⁷An equivocation is thus a pair of different (valid) messages (h, b) and (h', b') with $\text{LID}(h) = \text{LID}(h')$.

⁸Note that one can easily generalize this to use priorities that are an arbitrary function of the entire header.

1. The sender of a message—in addition to sending the header to its neighbors—also announces when it is in possession of the corresponding body.
2. When a party learns of a body announcement corresponding to the *preferred* header, which is the *first* header it has seen for a particular message ID, it downloads that body from the sender of the announcement and in turn announces possession of the body to its neighbors. The party then outputs the body to the environment. Importantly, the parties download bodies belonging to *higher-priority* headers first.

The network functionality. Based on the motivation above, F_{diff} (cf. Figure 1) allows for the transmission of messages consisting of a header and a body. Due to their small size, headers are delivered to all parties within a fixed delay

$$\Delta_{\text{hdr}} \text{ after sending}$$

(or after the first honest party has seen them). Also, F_{diff} guarantees that if one honest party sees a PoE, all other honest parties see it at most Δ_{hdr} later.

The delivery time of the (much larger) bodies, which for simplicity are all assumed to have the same size B , depends on the capacity and delays of the parties' connections, and on the traffic in the network. Specifically, consider a body b belonging to a header for which there are no equivocations. If there are no higher-priority messages impeding the diffusion of the body, it will be delivered $\beta := d \cdot (\frac{B}{C} + D)$ time after sending, where d is the diameter of the underlying network graph, C is the capacity of the connection between two peers, and D is an upper bound on the latency between two peers (imposed by the speed of light). The parameter C will be referred to as the “capacity of the network” in this work and represents the optimal throughput achievable in our model.⁹ For simplicity and in order for lower-level details such as the diameter d not to pertrude the model abstraction boundary, this expression is captured by a *base delay* β in F_{diff} .

Taking traffic into account, note that each body b' belonging to a higher-priority header will “cut in front” of b at most once, after which b continues its travel behind b' . The time b spends yielding to b' is at most the time it takes for b' to enter the connection, which is

$$\tau := B/C ,$$

where τ is referred to as the *queueing delay*.

⁹It is important to note, however, that in the type of gossip networks that motivate our design of F_{diff} , the network capacity C is *not* equivalent to the upload and download speed C' of individual parties. This discrepancy arises because, in a straightforward implementation of gossip, parties divide their bandwidth equally among their peers. As a result, $C = C'/\delta$, where δ represents the maximum degree in the network. It is conceivable that a more sophisticated gossip design could enable protocols that achieve throughputs up to C' .

Functionality F_{diff}

Parameters & Notation: $\Delta_{\text{hdr}} \in \mathbb{N}$, header propagation delay; β : base delay; B : body size; C , capacity of the network; $\text{match}(h, b)$, true iff body b matches header h ; $\text{LID}(h)$, lottery ID of h . $\mathbb{T}$ denotes the current time.

Admin

Variables: The functionality keeps track of the following variables, initialized to the values below:

- $\text{Hdrs} := \emptyset$: contains a triple $(h, t, \mathcal{P})$ for each header h diffused, where t is the time of diffusion and $\mathcal{P}$ the set of parties who have already received it.
- $\text{prefHdr}_P[\text{lid}] := \perp$: preferred header of P for message with lottery ID lid .
- $\text{Bdys} := \emptyset$: contains a tuple $(h, b, t, \mathcal{P})$ for header h and b diffused, where t is the time of diffusion and $\mathcal{P}$ the set of parties who have already received it.

Helpers: The description uses the following helpers:

- $\text{hasHdr}(P, \text{lid})$: returns true iff there exists $(h, \cdot, \mathcal{P}) \in \text{Hdrs}$ with $\text{LID}(h) = \text{lid}$ and $P \in \mathcal{P}$.
- $\text{hasPoE}(P, \text{lid})$: returns true iff there exist $(h, \cdot, \mathcal{P}), (h', \cdot, \mathcal{P}') \in \text{Hdrs}$ with $\text{LID}(h) = \text{LID}(h') = \text{lid}$ and $P \in \mathcal{P} \cap \mathcal{P}'$.
- $\text{hasBdy}(P, \text{lid})$: returns true iff there exists $(h, \cdot, \cdot, \mathcal{P}) \in \text{Bdys}$ with $\text{LID}(h) = \text{lid}$ and $P \in \mathcal{P}$.
- $\text{HdrsAdd}(h, t, P)$: if there exists $(h, \cdot, \mathcal{P}) \in \text{Hdrs}$, add P to $\mathcal{P}$; otherwise, add $(h, t, \{P\})$ to Hdrs .
- $\text{BdysAdd}(h, b, t, P)$: if there exists $(h, \cdot, \cdot, \mathcal{P}) \in \text{Hdrs}$, add P to $\mathcal{P}$; otherwise, add $(h, b, t, \{P\})$ to Bdys .
- $\text{higherBdys}(h)$: returns the number lid for which there exists $(h', \cdot, \cdot, \cdot) \in \text{Bdys}$ with $\text{LID}(h') > \text{LID}(h)$.

Diffusion

Full block: Upon (DIFFFB, h, b) from honest P :

1. Require: $\text{match}(h, b)$ and $\neg \text{hasHdr}(P, \text{LID}(h))$.

2. Bookkeeping: $\text{HdrsAdd}(h, \mathbb{T}, P)$, $\text{prefHdr}_P[\text{LID}(h)] := h$, and, if b is non-empty, $\text{BdysAdd}(h, b, \mathbb{T}, P)$.

Header: Upon $(\text{DIFFHDR}, h)$ from honest P :

1. Require: $\neg \text{hasPoE}(P, \text{LID}(h))$.
2. Bookkeeping: $\text{HdrsAdd}(h, \mathbb{T}, P)$, and if $\text{prefHdr}_P(\text{LID}(h)) = \perp$, set it to h .

Fetching

Headers: Upon (FETCHHDRS) from honest P :

1. Initialize “to-be-delivered” set $D \leftarrow \emptyset$.
2. For all $(h, t, \mathcal{P}) \in \text{Hdrs}$ where $P \notin \mathcal{P}$:
 - (a) If $\mathbb{T} \geq t + \Delta_{\text{hdr}}$ and $\neg \text{hasPoE}(P, \text{LID}(h))$, add h to D .
3. Output $(\text{FETCHHDRS}, P)$ to $\mathcal{A}$ and get a set $D_{\mathcal{A}}$ from $\mathcal{A}$.
4. Remove from $D_{\mathcal{A}}$ all h with $\text{hasPoE}(P, \text{LID}(h))$.
5. Output $(\text{FETCHHDRS}, D \cup D_{\mathcal{A}})$ to P .

Bodies: Upon (FETCHBDYS) from honest P :

1. Initialize “to-be-delivered” set $D \leftarrow \emptyset$.
2. For all $(h, b, t, \mathcal{P}) \in \text{Bdys}$ with $P \notin \mathcal{P}$:
 - (a) Add (h, b) to D if the following conditions are met:
 - i. $\text{prefHdr}_P[\text{LID}(h)] = h$;
 - ii. for all honest P' : $\text{prefHdr}_{P'}[\text{LID}(h)] \in \{h, \perp\}$;
 - iii. there exists $k \geq 0$ s.t. (a) $\mathbb{T} \geq t + \beta + k \cdot B/C$ and (b) $\text{higherBdys}(h) \leq k$.
3. Output $(\text{FETCHBDYS}, P)$ to $\mathcal{A}$ and get a set $D_{\mathcal{A}}$ from $\mathcal{A}$.
4. For each $(h, b) \in D_{\mathcal{A}}$, if (a) $\text{match}(h, b)$, (b) $\neg \text{hasBdy}(P, \text{LID}(h))$, and (c) $\text{prefHdr}_P(\text{LID}(h)) = h$: add (h, b) to D .
5. For each $(h, b) \in D$, $\text{BdysAdd}(h, b, \mathbb{T}, P)$.
6. Output $(\text{FETCHBDYS}, D)$ to P .

Fig. 1. Network functionality F_{diff} capturing the QUEQ model.

Based on the above, b will be delivered to all parties

$\beta + k \cdot \tau$ time after sending

(or after the first honest party has seen it) if at that point at most k higher-priority messages have been submitted to the network. Of course, the adversary may cause a message to be delivered at any time before.

Note that for equivocated messages, the body delivery guarantees are limited: since in the motivating gossip protocol above, parties only download bodies for their preferred headers, global body delivery is only guaranteed if all honest parties have the same preferred header. It is up to the application using F_{diff} to ensure that such messages are “not essential”—possibly by using PoEs.

4 Simplified Leios

The Leios protocol is a black-box construction of a high-throughput ledger protocol on top of a low-throughput protocol, the *base protocol* Π_{base} . The base protocol is assumed to satisfy persistence (cf. Definition 1) and ledger quality (cf. Definition 3). Moreover, it is assumed that the size of payload vectors supported by Π_{base} is large enough to contain the messages the overlay protocol must put on the base ledger.

The bandwidths for the base protocol and the Leios extension are separated: Specifically, the base protocol may require arbitrary hybrid functionalities $\mathcal{F}_i$ to run, including one that models the networking assumptions used by the base protocol. Conversely, functionality F_{diff} introduced in Section 3 is used to model the networking assumptions made by the Leios extension, which is designed to be near-optimal in the bandwidth C allocated to F_{diff} . Nevertheless, since the bandwidth needed for the base protocol is fixed, one can still achieve $1 - \delta$ throughput relative to large enough overall capacity.

This section presents *Simplified Leios*, a version of Leios that (1) achieves near-optimal throughput that is scaled by the ledger quality η of the base protocol and (2) has a somewhat undesirable trade-off between liveness delay and error probability. In rough terms, for parameters L and λ , Simplified Leios achieves¹⁰

- a relative (w.r.t. C) throughput of roughly $\frac{\lambda}{\lambda+1} \alpha_H \min(L/\eta, 1)$,
- settlement and liveness guarantees equal to the maximum of those of the base protocol and $O(\lambda^3 L)$,
- security error $\mathcal{O}(t)e^{-\Omega(L)}$, where t is an upper bound on the amount of time the protocol is run, and
- fairness.

Section 5 shows an augmented version, *Full Leios*, which achieves near-optimal throughput independent of η and with security error independent of L , meaning liveness delay can be constant (by choosing constant λ and L) while still yielding negligible error. For simplicity, both versions of Leios are first presented assuming that the adversary does not attempt to equivocate any protocol messages. Section 6 shows how to remove this assumption.

¹⁰Recall that α_H denotes the fraction of honest stake.

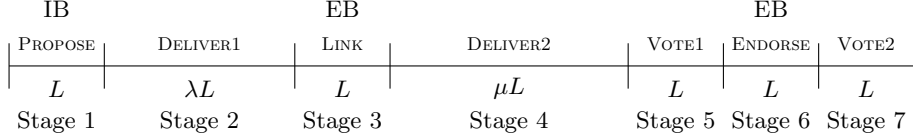


Fig. 2. The seven stages of the endorsing pipeline.

4.1 Protocol Overview

Input blocks. Leios is based on the concept of *input endorsing* [29,51,39], in which parties (run local, VRF-based lotteries to obtain the rights to) produce payload-carrying so-called *input blocks (IBs)*, which are then ordered using an underlying (low-throughput) consensus protocol. While the input-endorser concept is natural, implementing this technique, is non-trivial and requires considerable care, particularly in a realistic network model

One issue is that IBs can only be included in the underlying low-throughput ledger *by reference*. However, this leads to a data availability issue since the adversary may include references for which there are no corresponding IBs. Such behavior can be foiled by certifying IB availability via voting and modifying the ledger rules of the base protocol to only accept certified IBs.

Endorser blocks. Unfortunately, due to the high rate at which IBs need to be produced in order to achieve large throughput, certifying each IB individually would require too much computational overhead as well as space in the ledger of the (low-throughput) base protocol. To that end, IBs are first collected (by reference) in so-called *endorser blocks (EBs)* (also produced based on local, VRF-based lotteries), which are then certified. The idea is that parties contribute to the certification of an EB if they have seen all IBs referenced therein.

Fairness. A blockchain protocol is *fair* if the fraction of honestly created blocks in the chain corresponds to the actual fraction of honest stake (or whatever other resource underlies the security of the protocol). This extends to the input endorser technique in the sense that the fraction of honest IBs ultimately included via the base protocol should correspond to the fraction of honest stake.

The adversary may negatively affect the fairness of an input-endorser protocol by only referencing adversarially generated IBs in adversarial EBs. For example, if EBs are produced at a rate equal to the capacity of the base ledger, then such an adversarial strategy leads to roughly 75% adversarial IBs in settings where the adversarial stake is only slightly below 50%. It is therefore necessary to produce EBs at a rate high enough such that there is at least one honest EB for every block in the base protocol.

However, at such a high rate, not all EBs can be included in the low-throughput base protocol, and including only a subset of all EBs will not solve the fairness problem since it is unclear which EBs are from honest parties. Leios tackles this issue by suitably referencing older EBs from within newer EBs.

Freshest-first delivery. As explained in Section 1, IBs are diffused in a *freshest-first* fashion in order to prevent the attacker from spamming the network by releasing old IBs in bulk, i.e., to defend against message bursts. This is modeled by using time as the priority notion in F_{diff} (cf. Section 3). This approach comes with its own challenges, however, as special care has to be taken to ensure that IBs arrive before certain deadlines.

The pipeline. The Leios design is a pipelined one. A pipeline instance of Simplified Leios consists of seven stages (cf. Figure 2). Pipeline instances run concurrently, with a new instance beginning every L slots, where L referred to as the *slice length*. To accommodate our equivocation-handling mechanism (cf. Section 6), we require that $L \geq 5\Delta_{\text{hdr}}$ (where Δ_{hdr} is the header delay in F_{diff}). IBs and EBs are generated at a steady rate throughout the execution, where each IB plays a “proposer” role in the most recent pipeline instance, and each EB plays a “linking” role and an “endorsing” role in two different pipeline instances. The pipeline stages, whose lengths (in slots) are multiples $\{1, \lambda, \mu\}$ of L (where μ is another parameter defined later), are as follows:

1. PROPOSE (length L): IBs produced in this stage are the “focus” of the pipeline instance;
2. DELIVER1 (length λL): time for IBs from the PROPOSE stage to be diffused;
3. LINK (length L): IBs from the PROPOSE stage are referenced by EBs produced in this stage;
4. DELIVER2 (length μL): time for adversarial IBs referenced by the EBs from the LINK stage to diffuse;
5. VOTE1 (length L): parties cast votes for EBs from the LINK stage whose referenced IBs have been delivered; if an EB obtains a number of VOTE1-votes above a certain threshold, it becomes VOTE1-certified;
6. ENDORSE (length L): EBs from the LINK stage that have garnered a VOTE1-certificate are referenced by EBs from this stage;
7. VOTE2 (length L): parties cast votes for EBs from the ENDORSE stage that satisfy the following property: the EB references (i) VOTE1-certified EBs only and (ii) a majority of all VOTE1-certified LINK-stage EBs; if an EB obtains a number of VOTE2-votes above a certain threshold, it becomes VOTE2-certified.

Pipeline guarantees. The goal of every pipeline is to produce at least one VOTE2-certified EB while ensuring that each VOTE2-certified EB, via references to VOTE1-certified EBs, covers all honest IBs from the PROPOSE stage. Including any such EB per pipeline into the underlying low-throughput protocol will thus ensure liveness of honest input blocks as well as fairness.

Delivering IBs in time for EB inclusion. The (honest) IBs generated in the PROPOSE stage must be delivered to EB producers before the LINK stage begins.

To that end, PROPOSE is followed by a DELIVER1 stage, in which these IBs propagate through the network. However, the DELIVER1 stage overlaps with the PROPOSE stages of subsequent pipeline instances, and the IBs produced there take precedence because F_{diff} delivers IBs in a *freshes-first* manner. To guarantee that this interference does not delay the original IBs by too much, the overall IB rate must be such that the DELIVER1 stage, which is of length λL , can handle the IB traffic originating during both the PROPOSE and DELIVER1 stages. This is ensured by bounding the IB data rate (relative to the capacity of the network) by $\lambda/(\lambda + 1)$.

Note that thus, there is a tradeoff between higher throughput (larger λ) and shorter pipeline (smaller λ). The error $e^{-\Omega(L)}$ stems from a Chernoff bound guaranteeing that the number of IBs is close to its expectation.

Delivering IBs in time for first EB voting. Observe that honest EBs produced during LINK may only garner enough votes in VOTE1 if all the IBs they reference are globally delivered before VOTE1 starts. While honest IBs are guaranteed to be globally delivered already before LINK (as argued above), the adversary may start diffusing all of its IBs from PROPOSE late, in the worst case just before the end of DELIVER1. As a consequence, some honest EB producers may include those adversarial IBs, and therefore a second delivery period, DELIVER2, is required for them to be diffused globally before the start of VOTE1 as otherwise, these honest EBs may be at risk of being lost due to not receiving enough votes.

In a vein similar to DELIVER1 above, the delivery of said adversarial IBs is ensured if the (relative) IB data rate is below $(\mu + 1)/(\lambda + \mu + 2)$, which essentially guarantees that all the IB traffic of the first four stages fits into the LINK and DELIVER2 stages. In order not to further restrict the throughput, one requires $\lambda/(\lambda + 1) \leq (\mu + 1)/(\lambda + \mu + 2)$, which is satisfied for $\mu = \Theta(\lambda^2)$.

All honest EBs certified in time. The above guarantees that all honest EBs produced in LINK will be (globally seen as) VOTE1-certified before the beginning of the ENDORSE stage (assuming $L \geq \Delta_{\text{hdr}}$ —implied by the above assumption that $L \geq 5\Delta_{\text{hdr}}$). They will therefore be referenced by all honest EBs produced in ENDORSE.

All honest ENDORSE EBs will be VOTE2-certified since (i) they only reference VOTE1-certified LINK EBs and (ii) they all reference a majority of VOTE1-certified LINK EBs since all honest LINK EBs are certified as argued above (and honest parties control the majority of stake).

Therefore, by the end of the pipeline, there will be at least one VOTE2-certified EB that, via references to VOTE1-certified EBs, covers all honest IBs from the PROPOSE stage.

Actual availability of the payload. One final time, the adversary may introduce a delay, this time on the availability of the payload, by adopting the following (worst-case) strategy: just before the end of DELIVER2, it serves some IBs (that it was supposed to diffuse during PROPOSE), along with an EB that references them, to a small subset of honest parties, just enough to obtain sufficiently many

VOTE1 votes to get the EB VOTE1-certified. The EB will subsequently be picked up by honest EBs in the ENDORSE stage, and therefore the adversarial IBs end up referenced in the base ledger. However, they still need to be diffused to most honest parties, while also competing with newer IBs.

In the above spirit, these IBs will be delivered after an imaginary Stage 5' of length νL if the (relative) IB data rate is below $\nu/(2 + \lambda + \mu + \nu)$. Again, in order not to further restrict the throughput, one requires $\lambda(\lambda + 1) \leq \nu/(2 + \lambda + \mu + \nu)$, which is satisfied for $\nu = \Theta(\lambda^2 + \lambda\mu) = \Theta(\lambda^3)$.

Ledger construction. To extract the ledger, a party can parse the ledger of the base chain, and list all referenced (VOTE2-certified) EBs by order of appearance; from this list, the party can extract a list of LINK-stage EBs (also in order of appearance), and finally sequence all IBs referenced therein, finally defining an ordered list of ledger transactions.

While deriving the ledger in this fashion, some transactions may become invalid in the context of its preceding transactions and must therefore be dropped. Also, since the transaction order is not completely known before settlement, generally, all computation required for transaction validation needs to be done when a new base block settles that references the transaction for inclusion. In particular, this holds for account-based systems, e.g., Ethereum's EVM—although concurrent smart-contract execution [23] may be able to alleviate some of this cost. Interestingly, this bottleneck can be avoided in UTxO-based ledgers (e.g., Bitcoin, or Cardano, using the EUTxO model [17]). In such ledgers, verifying transaction correctness essentially consists of two parts: (1) check that all transaction inputs exist (and are unspent), and (2) that given those inputs, the script outcome is computed correctly. While the first (cheap) part of this computation can only be done once the serialization is complete, the second (expensive) part can be done “optimistically” prior to serialization—in our context during IB validation, i.e., the bulk part of validation is evenly spread over time alongside with IB production, without causing any computational spikes on the client side.

4.2 Protocol Messages

It will be useful to define a function that returns the slice x slices before the slice that contains some slot s . Specifically, the function returns the interval

$$\text{slice}_L(s, x) := [s', s' + (L - 1)] ,$$

for $s' := \lfloor s - xL \rfloor_L$, where $\lfloor \cdot \rfloor_L$ rounds down to the nearest multiple of L .

Input blocks. An input block $\text{IB} = (h, b)$ consists of a header h and a body b . The header $h = (\text{“IB”}, s, P, \pi, x, \sigma)$ consists of a slot number s , a stake holder ID P , a lottery proof π , a body hash x , and a signature σ on h (without σ). The body contains a list L_T of ledger transactions.

For the purpose of sending them via F_{diff} , set $\text{LID}(h) := (\text{“IB”}, s, P)$ —which also makes the lottery identifiers totally ordered—and $\text{match}((\cdot, \cdot, \cdot, \cdot, x, \cdot), b)$ if and only if x is the hash of b .

IBs are created at rate f_I , for some parameter f_I : given relative stake α , for a given slot, a stakeholder P is assigned the right to produce $\phi_I(\alpha) = \alpha f_I$ input blocks in expectation.¹¹

An input block is *valid* if lottery proof π is valid for (s, P) and signature σ is valid for P .¹²

Endorser blocks. Endorser blocks $EB = ("EB", s, P, \pi, L_I, L_E, \sigma)$ consist of a slot number s , a stake holder ID P , a lottery proof π , lists L_I and L_E of IB and EB references, respectively, and a signature σ on EB (without σ). Due to their small size, especially compared to IBs, EBs are modeled as “header-only” for the purpose of sending them via F_{diff} , where $\text{LID}(h) := ("EB", s, P)$.

EBs are created at rate f_E , for some parameter f_E : given relative stake α , for a given slot, a stakeholder P is assigned the right to produce $\phi_E(\alpha) = \alpha f_E$ endorser blocks in expectation.¹³

An EB is *valid* if (a) lottery proof π is valid for (s, P) and signature σ is valid for P , (b) L_I only references IBs from $\text{slice}_L(s, \lambda + 1)$, and (c) L_E only references EBs from $\text{slice}_L(s, \mu + 2)$. Note that EB validity can only be determined once all referenced IBs and EBs have been received.

Vote messages. Parties use $\mathcal{F}_{\text{sbv}}$ to generate voting strings for messages in different “periods” j . The periods used in Leios have the format $j = (s, x)$, where $x \in \{1, 2\}$, indicating whether the vote is for VOTE1 or VOTE2-certification.¹⁴ Intuitively, the $\mathcal{F}_{\text{sbv}}$ guarantees that (i) if all honest parties are instructed to vote for a particular message m in a period j , enough votes are generated to create a valid period- j certificate, and (ii) if no honest party votes for m in period j , the adversary cannot create a period- j certificate for m . Note that $\mathcal{F}_{\text{sbv}}$ can output $\perp$ to a party P in a period j , which means that P is not to cast a vote message in period j .

A vote message has the format $(\text{"vote"}, j, P, h, v)$, where j is a period number, P is stake holder ID, h is an EB reference, and $v \neq \perp$ is vote string output by $\mathcal{F}_{\text{sbv}}$ for message h and period j .¹⁵ Similarly to EBs, votes are also modeled as “header-only” for the purpose of sending them via F_{diff} , with $(\text{"vote"}, j, P)$ as the lottery ID.

It is important to note that, in contrast to IBs and EBs, which are produced over the course of the respective stage, all votes of a given stage (VOTE1 and VOTE2) are initiated right at the beginning of the stage.¹⁶

¹¹ For values of $f_I > 1$, one needs to subdivide each slot into $q > f_I$ subslots, each with $\phi_I(\alpha) = \alpha f_I / q$.

¹² Note that x being the hash of b is already guaranteed by F_{diff} with the above **match** predicate.

¹³ Similar to footnote 11.

¹⁴ Thus, the periods are totally ordered.

¹⁵ Note that v is tied to j , P , and h by $\mathcal{F}_{\text{sbv}}$; hence, no additional signature on the vote message is needed.

¹⁶ Possibly delaying some stage-VOTE2 votes by Δ_{hdr} in case a respective EB is produced late in the ENDORSE stage.

Protocol $\pi_{\text{simpleLeios}}$

Parameters: L : slice length; λ, μ : stage-length multipliers for DELIVER1 and DELIVER2 stages, respectively; f_I, f_E : frequencies of IBs and EBs, respectively (implicit in this figure).

General

Initialization: Upon receiving (INIT, V), store V and create and register the IB/EB lottery and voting keys with the key registration functionality, which as a response outputs the full public key list. The list is used to generate predicate V_{chkCerts} . Send input (INIT, V_{chkCerts}) to the base protocol; receive (STAKE, SD) from the base protocol and store SD (to be used for the IB/EB lottery and voting).

Network and ledger: At the beginning of each slot:

1. Fetch new messages (IBs, EBs, and votes) from F_{diff} and discard invalid ones (cf. Section 4.2). Re-input valid messages into F_{diff} (cf. Section 3).
2. Compute the ledger by following the EB-(EB-)*-IB references in the base ledger in order and concatenating the payloads in the IBs up to the point of the first IB with a non-downloaded body (if any); sanitize the ledger based on V , as detailed in Section 4.1.
3. Output the resulting ledger upon receiving command FTCHLDG.

Base chain: In each slot s (upon receiving (FTCHPLD) from the base protocol):

1. Issue (FTCHPLD) to the environment, receive (FTCHPLD, txs), and store transactions txs .
2. Pick an arbitrary VOTE2-certified EB EB from $\text{slice}_L(s, 2)$ and reply (FTCHPLD, EB) to the base protocol; if there are no such EBs, reply (FTCHPLD, txs) for the (most recently) stored txs .

Protocol Roles

In every slot s , each party P executes the following roles:

IB Role:

1. Check if eligible to produce an IB in slot s . If so, let π be the corresponding proof; otherwise, exit the procedure.
2. Issue (FTCHPLD) to the environment, receive (FTCHPLD, txs), and store transactions txs .
3. Set $IB := (h, \text{txs})$ where $h := ("IB", s, P, \pi, x, \sigma)$, where x is the hash of b and σ is a signature of (the rest of) h under P 's key in ML .
4. Diffuse IB via F_{diff} .

EB Role:

1. Check if eligible to produce an EB in slot s . If so, let π be the corresponding proof; otherwise, exit the procedure.
2. Link role: set L_I to be all (completely downloaded) IBs from $\text{slice}_L(s, \lambda + 1)$.
3. Endorse role: set L_E to be all VOTE1-certified EBs from $\text{slice}_L(s, \mu + 2)$.
4. Diffuse $EB := ("EB", s, P, \pi, L_I, L_E, \sigma)$ via F_{diff} , where σ is a signature of (the rest of) EB under P 's key in ML .

Vote-1 Role: If s is the first slot of $\text{slice}_L(s, 0)$:

1. Check if eligible to produce VOTE1-vote in slot s .
2. Create a vote for each EB from $\text{slice}_L(s, \mu + 1)$ for which all referenced IBs are fully downloaded.
3. Diffuse the votes via F_{diff} .

Vote-2 Role: If s is the first slot of $\text{slice}_L(s, 0)$:

1. Check if eligible to produce VOTE2-vote in slot s .
2. Create vote for each EB from $\text{slice}_L(s, 1)$ that references (i) a majority of all seen VOTE1-certified EBs from $\text{slice}_L(s, \mu + 3)$ and (ii) no other EBs.
3. Diffuse the votes via F_{diff} .

Fig. 3. The Simplified Leios Protocol.

Validity. In the following sections, it is tacitly assumed that parties drop all protocol messages that are not valid as defined in this section.

4.3 The Protocol

The Simplified Leios protocol is summarized in Figure 3. Initially, all parties locally generate keys for the IB/EB lottery as well as for voting, which they then register to the key registration functionality (cf. Section 2.1), that in turn distributes the public key list to all parties. Then, each party takes the voting public

keys registered and uses them to compute a verification function $V_{\text{chkCerts}}(\text{EB}, c)$ that accepts if and only if c is a valid certificate for EB under the voting keys. This predicate is then passed to the base protocol, which replies with the stake distribution to be used in the protocol.

Subsequently, in each slot, every party checks whether they are eligible to engage in any of the following roles:

- *IB role*: Creating IBs, corresponding to the PROPOSE stage.
- *EB role*: Creating EBs; this role can be subdivided into a link role and an endorse role, corresponding to the LINK and ENDORSE stages, respectively.
- *Vote-1 role*: Voting for EBs according to the VOTE1 stage.
- *Vote-2 role*: Voting for EBs according to the VOTE2 stage.

To illustrate this, assume $\lambda = \mu = 1$. In that case, there are seven active pipelines at any point, each of which is in a different stage. Assume they are numbered from 1 to 7 in ascending order of age (i.e., the newest one is pipeline 1). Given this, a party plays the following roles in each pipeline: *Pipeline 1*: IB role; *Pipeline 2*: no role; *Pipeline 3*: link role; *Pipeline 4*: no role; *Pipeline 5*: vote-1 role; *Pipeline 6*: endorse role; *Pipeline 7*: vote-2 role.

Parties continuously input a most recent VOTE2-certified endorser block to the base protocol, resulting in roughly every (η/L) -th pipeline being represented in the base chain. If there is no certified EB available (due to low participation—should the base protocol allow for dynamic participation), the parties input, as payloads to the base protocol, transactions received from the environment.¹⁷

4.4 Analysis Overview

This section provides an overview of the security analysis of Simplified Leios. The full analysis can be found in Appendix F. It is easy to see that under low participation, Leios inherits safety and liveness from the base protocol (because whenever there are no certified EBs available, parties input transactions directly to the base protocol); the remainder of this section therefore assumes full (honest) participation.

The main part of the analysis is to show that with high probability, a *pipeline instance* is *successful*, which intuitively means that (1) at least roughly $\alpha_H L f_1$ honest IBs are produced (during the PROPOSE stage),¹⁸ (2) at least one VOTE2-certified EB is produced, and (3) any VOTE2-certified EB references all honest IBs. Intuitively, (3) guarantees that roughly every (η/L) -th pipeline is represented in the base chain—as parties continuously input a most recent VOTE2-certified EB to the base protocol; (2) ensures that such an EB exists, and (1) ensures that enough honest IBs are available per pipeline to achieve the desired throughput.

The success of entire pipeline instances can be related to the success of each of the $5 + \lambda + \mu$ constituting slices; a *slice* is successful if (a) *at least* roughly $\alpha_H L f_1$

¹⁷We further expand on this idea in Appendix H.3.

¹⁸Recall that α_H denotes the fraction of honest stake.

honest IBs are produced, (b) *at most* roughly Lf_I honest IBs are produced, and (c) an honest majority of EBs are produced during the slice. Criterion (a) relates directly to (1); (b) ensures that the network is not too congested and the delivery guarantees of F_{diff} can be used to suitably bound IB delivery times, which is necessary for (2); and (c) helps guarantee (3). The probability $\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) = e^{-\varepsilon^2 \Omega(f_I L)} + e^{-\varepsilon^2 \Omega(f_E L)}$ that a slice is not successful is upper bounded using a standard Chernoff argument, where ε with $0 < \varepsilon < 2\alpha_H - 1$ is the concentration error in the Chernoff bound.

In order to show (2), the proof follows the intuition outlined in Section 4.1, which yields the following parameter choices:

$$f_I := \frac{\lambda - \rho}{\lambda + 1} \cdot \frac{C}{B} \quad \text{and} \quad \mu := \Theta(\lambda^2)$$

for $\rho := \beta/L$ (which becomes apparent in the analysis). Note that the above choice of f_I already suggests that the IB data rate $f_I B$ approaches C for large λ . Furthermore, also as explained in Section 4.1, the final liveness delay is dominated by $\nu := \mathcal{O}(\lambda^3)$.

Summarizing, this yields the theorems below, whose full proofs are provided in Appendix F. Here, let t be the time the protocol is run and $\varepsilon_{\text{vote}}(t)$ the error probability of the voting scheme if run for time t . Further, assume the base protocol satisfies persistence with parameter w and ledger quality η with liveness u ; let $\varepsilon_{\text{base}}(t)$ be the error of the base protocol when run for time t .

Theorem 1. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Then, Simplified Leios achieves persistence with parameter w' and liveness with parameters r' and u' , where $w' = w$, $r' = \eta + (6 + \lambda + \mu)L$, and $u' = u + (6 + \lambda + \mu + \nu)L$, except with probability $O(t)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) + \varepsilon_{\text{vote}}(t) + \varepsilon_{\text{base}}(t)$.*

Theorem 2. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Simplified Leios achieves $(\varepsilon_{\text{tp}}(t), R)$ -throughput for*

$$\varepsilon_{\text{tp}}(t) = O(t)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) + \varepsilon_{\text{vote}}(t) + \varepsilon_{\text{base}}(t)$$

and

$$R = \left(\frac{1 - \varepsilon}{1 + \varepsilon} \frac{\lambda - \rho}{\lambda + 1} \right) \min(L/\eta, 1) \cdot \alpha_H \cdot C.$$

The above implies that, with respect to the network capacity C , one can get *relative* throughput arbitrarily close to $\alpha_H L/\eta$ at the cost of (additional) latency $\mathcal{O}(\lambda^3 L) = \mathcal{O}(\gamma^{-3} L)$ for γ defined in Definition 4—where the additional latency is caused by the “detour” a payload needs to take via the endorsing pipeline; a detailed calculation is provided in Appendix F.

5 Full Leios

The objective of Full Leios is to maintain high throughput independent of the ledger quality η of the base protocol. This is accomplished by modifying Simplified Leios (cf. Section 4) such that pipelines reference each other for a limited

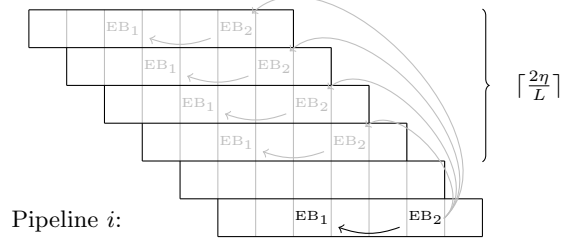


Fig. 4. Augmented endorsing pipeline. A new pipeline starts every L slots, and from the roughly $2\eta/L$ previous pipelines, all successful ones must be covered to guarantee full EB₂ coverage despite poor quality η of Π_{base} .

time horizon proportional to η , thus facilitating indirect ledger inclusion of all successful pipelines that emerged after the last honest payload entered the ledger.

This recursion is implemented by adapting the EB list $L_E = (L_{E,1}, L_{E,2})$ so that it consists of two sublists $L_{E,1}$ and $L_{E,2}$, where $L_{E,1}$ takes on the role of L_E in Simplified Leios, and $L_{E,2}$ references EBs from previous pipelines. Concretely, the rules for the ENDORSE stage are adapted as follows (cf. Figure 4), where i is the index of the pipeline instance:

1. ENDORSE: endorser blocks from this stage reference
 - (a) in $L_{E,1}$, all VOTE1-certified EB from the LINK stage (as before), and
 - (b) in $L_{E,2}$, for each of the pipeline instances $i - \lceil 2\eta/L \rceil, \dots, i - 2$, one VOTE2-certified EB from its ENDORSE stage—if such an EB exists.

Note that pipeline recursion by EB skips one instance ($i - 1$). The reason for this is that re-inclusion needs to wait for VOTE2-certification of prior endorser blocks, which for the case of the immediate predecessor pipeline, happens during the same slice wherein said EB is being produced.

A single base-protocol payload can now include multiple pipelines by containing one single EB together with its certificate. Still, two problems need to be solved to guarantee complete ledger inclusion of all successful pipelines:

- Issue 1. The EB recursion skips one instance. Thus, in case of direct ledger inclusion of an EB of pipeline instance i , we need to make sure that the recursion span is large enough for the next EB in an honest payload to catch up on the inclusion of pipeline instance $i - 1$.
- Issue 2. The adversary may craft EBs that omit successful pipelines they are supposed to cover in order to censor their IB production. We thus need a mechanism to detect such behavior such that honest base-protocol inclusion of such EBs is avoided.

Concrete rules. We modify the VOTE2-stage checks and define a rule for direct ledger inclusion of an EB. Together, these changes address the two problems stated at the end of the previous paragraph.

Specifically, besides the VOTE2-stage checks performed in Simplified Leios (see Figure 3), an EB (assume it to belong to pipeline instance i) must satisfy the following property with respect to the voting party's view:

- for each index $j \in \{i - \lceil 2\eta/L \rceil, \dots, i - 2\}$, if a VOTE2-certified EB_j of that pipeline instance was locally seen Δ_{hdr} before the last slot of pipeline j , then EB must reference, in $L_{\text{E},2}$, a VOTE2-certified EB'_j of the same pipeline instance.

Finally, we define the rule for base-protocol inclusion of EBs. We say that an EB is *eligible* for inclusion, iff it satisfies the following properties:

- it is VOTE2-certified, and,
- no EB from the same pipeline instance i is *directly* included in the most recent base-protocol payload (last EB in $\tilde{L}$ as returned by (FTCHLDG)).

The rule now states that, in each slot, each party replies (to the FTCHPLD request) as the payload to the base ledger, the most recent *eligible* EB (together with its VOTE2-certificate).

The second EB eligibility property together with EB recursion span 2η address Issue 1, as becomes evident in the analysis below. The extra VOTE2-check addresses Issue 2—by enabling the honest payload producer to detect censorship.

Analysis. It is easy to see that the guarantees of Simple Leios remain intact for Full Leios. The full analysis of the additional guarantees of Full Leios can be found in Appendix G.

The key insights are: (1) The failure of a slice affects at most $\mathcal{O}(5 + \lambda + \mu) = \mathcal{O}(\lambda^2)$ pipelines as $\mu = \mathcal{O}(\lambda^2)$; thus, the expected fraction of unsuccessful pipelines is at most $\varphi_{\text{fail}} := \mathcal{O}(\lambda^2)\varepsilon_{\text{fail}}$, where $\varepsilon_{\text{fail}} := \varepsilon_{\text{fail}}(\varepsilon, f_{\text{I}}, f_{\text{E}}, L)$ is the probability of a slice being unsuccessful (cf. Section 4.4), for an ε satisfying $0 < \varepsilon < 2\alpha_{\text{H}} - 1$. (2) All successful pipelines are covered by the recursive inclusion.

This yields the theorems below. Here, let t be the time the protocol is run and $\varepsilon_{\text{vote}}(t)$ the error probability of the voting scheme if run for time t . Further, assume the base protocol satisfies persistence with parameter w and ledger quality η with liveness u ; let $\varepsilon_{\text{base}}(t)$ be the error of the base protocol when run for time t . Finally, let ρ as in Section 4.4.

Theorem 3. *Full Leios achieves persistence with parameter $w' = w$, and, on average (over all transactions), the protocol achieves liveness with parameters $r' = L + \frac{L}{1 - \varepsilon_{\text{fail}}}$, and $u' = \frac{L}{1 - \varepsilon_{\text{fail}}} + \mathcal{O}(\lambda^3)L + \eta + u$, except with probability $\varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$.*

Theorem 4. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Full Leios achieves $(\varepsilon_{\text{tp}}(t), R)$ -throughput for $\varepsilon_{\text{tp}}(t) = \varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$ and $R = \left(\frac{1-\varepsilon}{1+\varepsilon} \frac{\lambda-\rho}{\lambda+1}\right) (1 - \varphi_{\text{fail}})\alpha_H C$ with $\varphi_{\text{fail}} = \mathcal{O}(\lambda^2)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$.*

The above implies that, with respect to the network capacity C , one can get a *relative* throughput arbitrarily close to $(1 - \varphi_{\text{fail}})\alpha_H$ at the cost of (additional) latency $\mathcal{O}(\gamma^{-3} \log \gamma^{-1})$ for γ defined in Definition 4—except for negligible error in the security parameter; a detailed calculation is provided in Appendix G.

Finally, as φ_{fail} can be made arbitrarily small, we get the following theorem which is also proven in Appendix G.

Theorem 5. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Full Leios achieves $(\varepsilon_{\text{tp}}(t), R)$ -throughput for $\varepsilon_{\text{tp}}(t) = \varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$ and $R = \left(\frac{1-\varepsilon}{1+\varepsilon} \frac{\lambda-\rho}{\lambda+1}\right) \alpha_H C$ which is near-optimal.*

6 Equivocation Handling

There are three types of protocol messages in Leios: input blocks (IBs), endorser blocks (EBs), and votes. All of these messages are diffused via F_{diff} , which means that by introducing equivocations, the adversary can split the views of honest parties in terms of which their preferred message is for a given lottery ID. Therefore, for each message type, such *splits* must be dealt with by an explicit strategy.

This section presents a strategy that guarantees that for each block/vote opportunity in the Leios overlay protocol,

- each honest party downloads and forwards *at most two block headers*,¹⁹
- each honest party downloads *at most one block body*, and
- only the fully downloaded block/vote is required for the protocol (and, in particular, ledger correctness).

The strategy presented below assumes that the LINK stage does not start until $5\Delta_{\text{hdr}}$ after the last IB production opportunity in the PROPOSE stage, which is the case if $\lambda L \geq 5\Delta_{\text{hdr}}$.

Input blocks. Observe that for each block opportunity, F_{diff} delivers to a party P at most one body, namely the body belonging to the first header received by P ;²⁰ below, the actual block downloaded by P is called the *preferred block for P* . First, one has to ensure that EBs are only voted for if for *all* referenced IBs $\text{IB} = (h, b)$, all honest parties have the same preferred IB for $\text{LID}(h)$. This is achieved by adding an additional check to the VOTE1-rule:

¹⁹Recall that due to their small size, EBs and votes are modeled as “header-only,” i.e., they have an empty body, whereas an IB consists of a header and a body, where the body contains the actual payload. Downloading a *bounded* number of headers is acceptable since they are rather small compared to bodies.

²⁰As explained in Section 3 this behavior is straightforward to implement in the underlying gossip protocol.

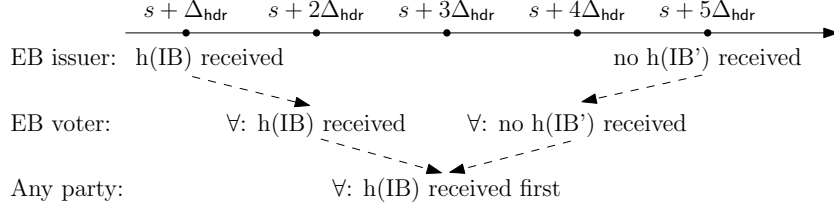


Fig. 5. Equivocation handling for input blocks. $h(\text{IB})$ stands for “header of IB.”

- A voter P in the VOTE1 stage only votes for an EB if *all* referenced IBs $\text{IB} = (h, b)$ satisfy: P received (a) h before time $s + 2\Delta_{\text{hdr}}$ and (b) no equivocating header h' before time $s + 4\Delta_{\text{hdr}}$, where s is the slot number of h .

The $2\Delta_{\text{hdr}}$ gap between criteria (a) and (b) ensure that no honest party P' has a preferred header different from P : Assume some P' prefers a different header h' . Then, P' would have to have received h' before $s + 3\Delta_{\text{hdr}}$ as h or a proof of equivocation reach P' at $s + 3\Delta_{\text{hdr}}$. In either case, P would see a proof of equivocation before $s + 4\Delta_{\text{hdr}}$. Hence, an EB is only voted for by honest parties if there is no split for any IBs it references.

It remains to determine, for EB producers in the LINK stage, an IB-inclusion strategy that guarantees that their EBs are voted for based on the above voting rule:

- An EB producer P in the LINK-stage only references $\text{IB} = (h, b)$ in list L_1 if P received (i) h before time $s + \Delta_{\text{hdr}}$, and (ii) no equivocating header h' before time $s + 5\Delta_{\text{hdr}}$.

Criterion (i) guarantees that criterion (a) is satisfied for the voters, and (ii) guarantees that (b) is satisfied for the voters. Hence, any honest EB from the LINK-stage will be voted for in the VOTE1 stage. The IB equivocation strategy is illustrated in Figure 5.

Endorser blocks (EB). Along the above lines, we need to ensure that a particular EB only becomes VOTE2-certified if EB is the preferred block for its respective block opportunity. Let $\text{EB} = (h)$ be an EB from some slot s ; recall that EBs consist of headers only. The timing rule is as follows: a voter P in the vote stage only votes for an EB if P received (a) h before time $s + \Delta_{\text{hdr}}$ and (b) no equivocating header h' before time $s + 3\Delta_{\text{hdr}}$. Note, that if any honest party voted in favor of a block, then it must be the case that this block is the preferred one for all honest parties.

Votes. Note that votes for a particular EB are deterministic, and cannot be equivocated. Still, for a given voting opportunity, the adversary may create votes for different (equivocated) blocks of the same block opportunity. This problem can be easily mitigated by having parties only download (and diffuse) the votes corresponding to the preferred EB of a given block opportunity.

Analysis. We prove some simple facts about the above mechanism that can be used in a straight-forward way to extend the security theorems of Sections 4 and 5 to the setting where block equivocations are considered.

Lemma 1. *For each block opportunity, each honest party downloads at most two block headers and one block body.*

Proof. The lemma directly follows by construction. $\square$

Lemma 2. *If EB is voted for by an honest party, then all IBs referenced in EB are downloaded by all honest parties.*

Proof. See Fig. 5. An honest party only votes for EB if the header of each referenced IB was received by time $s + 2\Delta_{\text{hdr}}$ where s is the slot of IB, and no equivocating header for any such IBs was received by time $s + 4\Delta_{\text{hdr}}$ (middle row). This implies that, for each referenced IB, all honest parties received the header by time $s + 3\Delta_{\text{hdr}}$ as the first header seen for the respective block opportunity (bottom row). All honest parties thus ‘prefer’ and download all IBs referenced in EB. $\square$

Lemma 3. *If EB is voted for by an honest party, then EB gets downloaded by all honest parties.*

Proof. An honest party only votes for EB if the header of EB was received by time $s + \Delta_{\text{hdr}}$ where s is the slot of EB, and no equivocating header for EB was received by time $s + 3\Delta_{\text{hdr}}$. This implies that all honest parties received the header of EB by time $s + 2\Delta_{\text{hdr}}$ as the first header for the given block opportunity, and thus download EB. $\square$

Lemma 4. *Given that the respective pipeline is successful, every EB produced by an honest party gets voted for by each honest party.*

Proof. The IB timing rule for IB inclusion maintains enough slack to imply the necessary timing (see Fig. 5) for respective voting for every honest party. Similarly, the EB timing rule for voting is implied by the delivery guarantees for EB headers. $\square$

References

1. Al-Bassam, M., Sonnino, A., Buterin, V.: Fraud and data availability proofs: Maximising light client security and scaling blockchains with dishonest majorities. arXiv preprint arXiv:1809.09044 (2018)
2. Avarikioti, G., Käppeli, L., Wang, Y., Wattenhofer, R.: Bitcoin security under temporary dishonest majority. In: Goldberg, I., Moore, T. (eds.) FC 2019. LNCS, vol. 11598, pp. 466–483. Springer, Cham (Feb 2019). https://doi.org/10.1007/978-3-030-32101-7_28
3. Avarikioti, Z., Heimbach, L., Schmid, R., Vanbever, L., Wattenhofer, R., Wintermeyer, P.: Fnf-BFT: Exploring performance limits of bft protocols (2020)

4. Badertscher, C., Gaži, P., Kiayias, A., Russell, A., Zikas, V.: Consensus redux: Distributed ledgers in the face of adversarial supremacy. In: 2024 IEEE 37th Computer Security Foundations Symposium (CSF). pp. 267–282. IEEE Computer Society, Los Alamitos, CA, USA (jul 2024). <https://doi.org/10.1109/CSF61375.2024.00018>, <https://doi.ieeecomputersociety.org/10.1109/CSF61375.2024.00018>
5. Badertscher, C., Gaži, P., Kiayias, A., Russell, A., Zikas, V.: Consensus redux: Distributed ledgers in the face of adversarial supremacy. Cryptology ePrint Archive, Report 2020/1021 (2020), <https://eprint.iacr.org/2020/1021>
6. Badertscher, C., Gazi, P., Kiayias, A., Russell, A., Zikas, V.: Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In: Lie, D., Man- nan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 913–930. ACM Press (Oct 2018). <https://doi.org/10.1145/3243734.3243848>
7. Bagaria, V.K., Kannan, S., Tse, D., Fanti, G.C., Viswanath, P.: Prism: Decon- structing the blockchain to approach physical limits. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019. pp. 585–602. ACM Press (Nov 2019). <https://doi.org/10.1145/3319535.3363213>
8. Bellare, M., Miner, S.K.: A forward-secure digital signature scheme. In: Wiener, M.J. (ed.) CRYPTO’99. LNCS, vol. 1666, pp. 431–448. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_28
9. Buterin, V., Griffith, V.: Casper the friendly finality gadget. arXiv preprint arXiv:1710.09437 (2017)
10. Cachin, C., Tessaro, S.: Asynchronous verifiable information dispersal. In: 24th IEEE Symposium on Reliable Distributed Systems (SRDS’05). pp. 191–201. IEEE (2005)
11. Camenisch, J., Enderlein, R.R., Krenn, S., Küsters, R., Rausch, D.: Universal com- position with responsive environments. In: Cheon, J.H., Takagi, T. (eds.) ASI- ACRYPT 2016, Part II. LNCS, vol. 10032, pp. 807–840. Springer, Berlin, Heidel- berg (Dec 2016). https://doi.org/10.1007/978-3-662-53890-6_27
12. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145. IEEE (2001)
13. Castro, M., Liskov, B., et al.: Practical byzantine fault tolerance. In: OsDI. vol. 99, pp. 173–186 (1999)
14. Chaidos, P., Kiayias, A.: Mithril: Stake-based threshold multisignatures. Cryptol- ogy ePrint Archive, Report 2021/916 (2021), <https://eprint.iacr.org/2021/916>
15. Chaidos, P., Kiayias, A., Reyzin, L., Zinovyev, A.: Approximate lower bound ar- guments. Cryptology ePrint Archive (2023)
16. Chaidos, P., Kiayias, A., Reyzin, L., Zinovyev, A.: Approximate lower bound ar- guments. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part IV. LNCS, vol. 14654, pp. 55–84. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58737-5_3
17. Chakravarty, M.M.T., Chapman, J., MacKenzie, K., Melkonian, O., Jones, M.P., Wadler, P.: The extended UTXO model. In: Bernhard, M., Bracciali, A., Camp, L.J., Matsuo, S., Maurushat, A., Rønne, P.B., Sala, M. (eds.) FC 2020 Workshops. LNCS, vol. 12063, pp. 525–539. Springer, Cham (Feb 2020). https://doi.org/10.1007/978-3-030-54455-3_37
18. Croman, K., Decker, C., Eyal, I., Gencer, A.E., Juels, A., Kosba, A.E., Miller, A., Saxena, P., Shi, E., Sirer, E.G., Song, D., Wattenhofer, R.: On scaling decentralized blockchains - (A position paper). In: Clark, J., Meiklejohn, S.,

- Ryan, P.Y.A., Wallach, D.S., Brenner, M., Rohloff, K. (eds.) FC 2016 Workshops. LNCS, vol. 9604, pp. 106–125. Springer, Berlin, Heidelberg (Feb 2016). https://doi.org/10.1007/978-3-662-53357-4_8
19. Danezis, G., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A.: Narwhal and tusk: a dag-based mempool and efficient bft consensus. In: Proceedings of the Seventeenth European Conference on Computer Systems. pp. 34–50 (2022)
 20. Danezis, G., Meiklejohn, S.: Centrally banked cryptocurrencies. In: NDSS 2016. The Internet Society (Feb 2016). <https://doi.org/10.14722/ndss.2016.23187>
 21. David, B., Gazi, P., Kiayias, A., Russell, A.: Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part II. LNCS, vol. 10821, pp. 66–98. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78375-8_3
 22. Dembo, A., Kannan, S., Tas, E.N., Tse, D., Viswanath, P., Wang, X., Zeitouni, O.: Everything is a race and nakamoto always wins. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020. pp. 859–878. ACM Press (Nov 2020). <https://doi.org/10.1145/3372297.3417290>
 23. Dickerson, T.D., Gazzillo, P., Herlihy, M., Koskinen, E.: Adding concurrency to smart contracts. In: Schiller, E.M., Schwarzmann, A.A. (eds.) 36th ACM PODC. pp. 303–312. ACM (Jul 2017). <https://doi.org/10.1145/3087801.3087835>
 24. Dubhashi, D.P., Panconesi, A.: Concentration of measure for the analysis of randomized algorithms. Cambridge University Press (2009)
 25. Eyal, I., Gencer, A.E., Sirer, E.G., van Renesse, R.: Bitcoin-ng: A scalable blockchain protocol. In: Argyraki, K.J., Isaacs, R. (eds.) 13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16–18, 2016. pp. 45–59. USENIX Association (2016), <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eyal>
 26. Eyal, I., Sirer, E.G.: Majority is not enough: Bitcoin mining is vulnerable. In: Christin, N., Safavi-Naini, R. (eds.) FC 2014. LNCS, vol. 8437, pp. 436–454. Springer, Berlin, Heidelberg (Mar 2014). https://doi.org/10.1007/978-3-662-45472-5_28
 27. Fitzi, M., Gazi, P., Kiayias, A., Russell, A.: Parallel chains: Improving throughput and latency of blockchain protocols via parallel composition. Cryptology ePrint Archive, Report 2018/1119 (2018), <https://eprint.iacr.org/2018/1119>
 28. Fitzi, M., Wang, X., Kannan, S., Kiayias, A., Leonardos, N., Viswanath, P., Wang, G.: Minotaur: Multi-resource blockchain consensus. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 1095–1108 (2022)
 29. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol: Analysis and applications. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part II. LNCS, vol. 9057, pp. 281–310. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46803-6_10
 30. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol with chains of variable difficulty. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 291–323. Springer, Cham (Aug 2017). https://doi.org/10.1007/978-3-319-63688-7_10
 31. Gazi, P., Kiayias, A., Russell, A.: Tight consistency bounds for bitcoin. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020. pp. 819–838. ACM Press (Nov 2020). <https://doi.org/10.1145/3372297.3423365>
 32. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: Proceedings of the 26th symposium on operating systems principles. pp. 51–68 (2017)

33. Gupta, S., Hellings, J., Sadoghi, M.: Rcc: Resilient concurrent consensus for high-throughput secure transaction processing. arXiv preprint arXiv:1911.00837 (2019)
34. Itkis, G., Reyzin, L.: Forward-secure signatures with optimal signing and verifying. In: Kilian, J. (ed.) CRYPTO 2001. LNCS, vol. 2139, pp. 332–354. Springer, Berlin, Heidelberg (Aug 2001). https://doi.org/10.1007/3-540-44647-8_20
35. Kalodner, H.A., Goldfeder, S., Chen, X., Weinberg, S.M., Felten, E.W.: Arbitrum: Scalable, private smart contracts. In: Enck, W., Felt, A.P. (eds.) USENIX Security 2018. pp. 1353–1370. USENIX Association (Aug 2018)
36. Keidar, I., Kokoris-Kogias, E., Naor, O., Spiegelman, A.: All you need is DAG. In: Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing. pp. 165–175 (2021)
37. Keidar, I., Naor, O., Poupko, O., Shapiro, E.: Cordial miners: Fast and efficient consensus for every eventuality. arXiv preprint arXiv:2205.09174 (2022)
38. Kiayias, A., Koutsoupias, E., Lazos, P., Panagiotakos, G.: Blockchain space tokenization. In: Böhme, R., Kiffer, L. (eds.) 6th Conference on Advances in Financial Technologies, AFT 2024, September 23-25, 2024, Vienna, Austria. LIPIcs, vol. 316, pp. 9:1–9:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2024). <https://doi.org/10.4230/LIPICS.AFT.2024.9>, <https://doi.org/10.4230/LIPICS.AFT.2024.9>
39. Kiayias, A., Russell, A., David, B., Oliynykov, R.: Ouroboros: A provably secure proof-of-stake blockchain protocol. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 357–388. Springer, Cham (Aug 2017). https://doi.org/10.1007/978-3-319-63688-7_12
40. Kiffer, L., Neu, J., Sridhar, S., Zohar, A., Tse, D.: Security of nakamoto consensus under congestion. Cryptology ePrint Archive, Paper 2023/381 (2023), <https://eprint.iacr.org/2023/381>, <https://eprint.iacr.org/2023/381>
41. Lamport, L., Shostak, R., Pease, M.: The byzantine generals problem. ACM Transactions on Programming Languages and Systems (TOPLAS) 4(3), 382–401 (1982)
42. Li, S., Yu, M., Avestimehr, A.S., Kannan, S., Viswanath, P.: PolyShard: Coded sharding achieves linearly scaling efficiency and security simultaneously. Cryptology ePrint Archive, Report 2018/925 (2018), <https://eprint.iacr.org/2018/925>
43. Liang, G., Vaidya, N.: Capacity of byzantine agreement with finite link capacity. In: 2011 Proceedings IEEE INFOCOM. pp. 739–747. IEEE (2011)
44. Micali, S., Rabin, M.O., Vadhan, S.P.: Verifiable random functions. In: 40th FOCS. pp. 120–130. IEEE Computer Society Press (Oct 1999). <https://doi.org/10.1109/SFFCS.1999.814584>
45. Micali, S., Reyzin, L., Vlachos, G., Wahby, R.S., Zeldovich, N.: Compact certificates of collective knowledge. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 626–641. IEEE (2021)
46. Milosevic, Z., Biely, M., Schiper, A.: Bounded delay in byzantine-tolerant state machine replication. In: 2013 IEEE 32nd International Symposium on Reliable Distributed Systems. pp. 61–70. IEEE (2013)
47. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. <http://bitcoin.org/bitcoin.pdf> (2008)
48. Nazirkhanova, K., Neu, J., Tse, D.: Information dispersal with provable retrievability for rollups. Cryptology ePrint Archive, Report 2021/1544 (2021), <https://eprint.iacr.org/2021/1544>
49. Neu, J., Sridhar, S., Yang, L., Tse, D., Alizadeh, M.: Longest chain consensus under bandwidth constraint. In: Proceedings of the 4th ACM Conference on Advances in Financial Technologies. pp. 126–147 (2022)

50. Pass, R., Seeman, L., shelat, a.: Analysis of the blockchain protocol in asynchronous networks. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part II. LNCS, vol. 10211, pp. 643–673. Springer, Cham (Apr / May 2017). https://doi.org/10.1007/978-3-319-56614-6_22
51. Pass, R., Shi, E.: FruitChains: A fair blockchain. In: Schiller, E.M., Schwarzmann, A.A. (eds.) 36th ACM PODC. pp. 315–324. ACM (Jul 2017). <https://doi.org/10.1145/3087801.3087809>
52. Pass, R., Shi, E.: Rethinking large-scale consensus. In: Köpf, B., Chong, S. (eds.) CSF 2017 Computer Security Foundations Symposium. pp. 115–129. IEEE Computer Society Press (2017). <https://doi.org/10.1109/CSF.2017.37>
53. Pass, R., Shi, E.: The sleepy model of consensus. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part II. LNCS, vol. 10625, pp. 380–409. Springer, Cham (Dec 2017). https://doi.org/10.1007/978-3-319-70697-9_14
54. Rabin, M.O.: Efficient dispersal of information for security, load balancing, and fault tolerance. *Journal of the ACM (JACM)* **36**(2), 335–348 (1989)
55. Spiegelman, A., Girdharan, N., Sonnino, A., Kokoris-Kogias, L.: Bullshark: Dag bft protocols made practical. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 2705–2718 (2022)
56. Stathakopoulou, C., Tudor, D., Pavlovic, M., Vukolić, M.: Mir-bft: Scalable and robust bft for decentralized networks. *Journal of Systems Research* **2**(1) (2022)
57. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing. pp. 347–356 (2019)
58. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing. pp. 347–356 (2019)
59. Yu, H., Nikolic, I., Hou, R., Saxena, P.: OHIE: Blockchain scaling made simple. In: 2020 IEEE Symposium on Security and Privacy. pp. 90–105. IEEE Computer Society Press (May 2020). <https://doi.org/10.1109/SP40000.2020.00008>
60. Zamani, M., Movahedi, M., Raykova, M.: RapidChain: Scaling blockchain via full sharding. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 931–948. ACM Press (Oct 2018). <https://doi.org/10.1145/3243734.3243853>
61. Zhang, R., Zhang, D., Wang, Q., Wu, S., Xie, J., Preneel, B.: NC-max: Breaking the security-performance tradeoff in nakamoto consensus. In: Proceedings 2022 Network and Distributed System Security Symposium. pp. 1–18. NDSS (2022)

A Additional Related Work

This section details additional related work.

More details on throughput limitations of conventional protocols. Low transaction throughput is inherent to “conventional” blockchain protocols (pure Nakamoto or BFT) since block production is strictly sequential, and a new block must reach the subsequent leader before yet another block can be published. This lag inevitably sacrifices throughput, since the network overall remains underutilized waiting for a single block to propagate to all nodes. To see this in some more detail, in Nakamoto consensus [47] average throughput is bounded by fB , where f is the expected number of blocks per second and B the size of a block in terms of transaction bytes. Based on the security analysis of the Bitcoin blockchain, [31,22], a tight bound for security is $f\Delta < (1/p_A - 1/p_H)$ where Δ is the block propagation delay, p_A the probability that the adversary obtains a PoW and p_H the same probability for the honest parties. Given that $\Delta \geq dB/C$ where d is the diameter of the network and C the capacity of a channel between two nodes in the network in bytes per second, it follows that throughput is bounded by $(1/p_A - 1/p_H) \cdot C/d$.²¹ It follows that (standard) Nakamoto consensus is suboptimal in terms of network utilization.

Sharding and layer-2 protocols. Blockchain sharding was introduced by Danezis and Meiklejohn [20] and Croman *et al.* [18]. In contrast to a fully replicated blockchain wherein each full node stores the complete ledger state, a sharded blockchain partitions the ledger state by assigning different committees who each maintain a different part of the ledger. Sharding could potentially enhance transaction throughput beyond the network’s capacity by avoiding complete ledger replication across all parties. Instead, specific parties are tasked with maintaining shards, including distributing shard data to the public. Layer-2 (L2) protocols, see e.g., [35,48] offer a different approach to scaling: they typically centralize off-chain processing and then via incentives and/or zero-knowledge proofs ensure that the off-chain execution is correct. While both sharding and L2 approaches can be fundamental facilitators of further scaling – they address a different problem compared to the question of scaling a blockchain (L1) protocol to its absolute physical limits — which is the focus of our work.

On some tools used by Leios. Data-availability proofs based on Rabin’s information dispersal [54] and the work by [10] have been widely used in the context of light clients or sharding (e.g., [60,1,42]). In contrast, Leios relies on signatures to ensure data availability since this is more amenable for our use in a gossip network.

Using pipelining to increase the throughput of BFT protocols has been proposed in [43] and was subsequently used in, e.g., [9,58]—which can be seen as an

²¹Note that, in the worst case, each block producer includes transactions that are not yet contained in the other parties’ mempools.

alternative way (to input endorsing) of decoupling payload dissemination from achieving consensus. Note that Leios pipelining is not directly related to that kind of pipelining since decoupling for the purpose of high throughput is already provided by input endorsing: in contrast, Leios pipelining is exclusively applied to provide timely-delivery guarantees for block-related data under freshest-first message delivery.

The idea of defending against message bursts by prioritizing fresher messages for download was introduced in [49] in form of downloading *towards* the freshest block in PoS Nakamoto consensus.

Equivocation proofs are (implicitly or explicitly) present in every Byzantine-Agreement/BFT protocol relying on digital signatures, dating all the way back to the initial work by Lamport *et al.* [41].

B Chernoff-Hoeffding bounds

We make use of the Chernoff-Hoeffding bound in form of Eq. (1.7) in [24].

Theorem 6 ([24]). *Let $X := \sum_{i \in [n]} X_i$ where X_i , $i \in [n]$ are independently distributed in $[0, 1]$. Then, for $0 < \varepsilon < 1$,*

$$\Pr[X > (1 + \varepsilon)E[X]] \leq \exp\left(-\frac{\varepsilon^2}{3}E[X]\right),$$

and

$$\Pr[X < (1 - \varepsilon)E[X]] \leq \exp\left(-\frac{\varepsilon^2}{2}E[X]\right),$$

C Verifiable Random Functions

A *verifiable random function (VRF)* is a cryptographic primitive that allows a party P to create a key pair consisting of a secret evaluation key and a public verification key such that: (a) the public key implicitly defines a function f (b) with the secret key, P can evaluate f at any input x , obtaining an output y and a proof π ; (c) given the public key of P , anyone is able to verify, using π , that y is indeed the output corresponding to x ; (d) with only the public verification key, outputs of the function cannot be predicted, and in fact appear random.

The above guarantees are abstracted by an idealized functionality F_{vrf} (cf. Figure 6). The main commands offered by F_{vrf} are (i) (GETKEY), answered by (GETKEY, v) for an (adversarially chosen) idealized public key v , (ii) (EVAL, v, x), answered by (EVAL, y, π), and (iii) (VERIFY, v, x, y, π), answered by (VERIFY, ϕ) with $\phi \in \{0, 1\}$ indicating whether π is a valid proof for the input/output pair (x, y) under public key v .

Functionality F_{vrf}

Variables: The functionality keeps track of the following variables, initialized to the values below:

1. an array $\text{Keys}[P] := \emptyset$: set of keys owned by P ;
2. an array $T[v, x] := \perp$, where v is a key and x a domain value: pair (y, S) , where y is a range value and S a set of proofs π ;
3. a set $E := \emptyset$: contains triples (v, x, y) to keep track of all VRF evaluations.

Keys: Upon (GETKEY) from P : Output (GETKEY, P) to $\mathcal{S}$ and ask $\mathcal{S}$ for a unique key v . Add v to $\text{Keys}[P]$ and output (GETKEY, v) to P .

Register Keys: Upon (REGKEY, v) from $\mathcal{S}$: if v is unique, add it to $\text{Keys}[\mathcal{S}]$.

Evaluation: Upon (EVAL, v, x) from P with $v \in \text{Keys}[P]$: Output (EVAL, P, v, x) to $\mathcal{S}$, and upon obtaining (EVAL, π) from $\mathcal{S}$:

1. If π is not unique, exit the procedure.
2. If $T[v, x]$ is undefined, pick y uniformly at random from the range and set $S := \emptyset$; otherwise, let $(y, S) := T[v, x]$.
3. Set $T[v, x] := (y, S \cup \{\pi\})$ and add (v, x, y) to E .
4. Output (EVAL, y, π) to P .

Malicious evaluation: Upon (EVAL, v, x) from $\mathcal{S}$:

- *Case 1:* There exists an *uncorrupted* P with $v \in \text{Keys}[P]$: If $(y, S) := T[v, x]$ is defined, return (EVAL, y) to $\mathcal{S}$; otherwise, do nothing.
- *Case 2:* There exists a *corrupted* P with $v \in \text{Keys}[P]$ or $v \in \text{Keys}[\mathcal{S}]$: If $T[v, x]$ is not defined, pick y uniformly at random from the range, let $S := \emptyset$ and set $T[v, x] := (y, S)$; otherwise, let $(y, S) := T[v, x]$. Return (EVAL, y) to $\mathcal{S}$.
- *Else:* Do nothing.

Verification: Upon (VERIFY, v, x, y, π) from any ITI: Send (VERIFY, v, x, y, π) to $\mathcal{S}$, and upon receiving (VERIFY, ϕ) from $\mathcal{S}$:

- If $v \in \text{Keys}[\cdot]$ and $T[v, x] = (y, S)$ is defined:
 1. If $\pi \in S$, set $f := 1$.
 2. Else, if $\phi = 1$ and π is unique—i.e., if for all $(v', x') \neq (v, x)$ with $T[v', x'] = (\cdot, S)$, $\pi \notin S$ —set $T[v, x] := (y, S \cup \{\pi\})$ and $f := 1$.
 3. Else, set $f := 0$.
- Else, set $f := 0$.

Output (VERIFY, f) to P .

Adversarial leakage: Upon (LEAK) from $\mathcal{S}$: return (LEAK, E) to $\mathcal{S}$.

Fig. 6. VRF functionality F_{vrf} .

D Key Evolving Signatures

A *key evolving signature (KES)* is essentially a forward-secure digital signature scheme. As in classical signatures, a party can create a key pair that consists of a secret signing key and a public verification key. To allow for forward security the secret key is updated/evolved every time a signature is produced (for a bounded number of updates), while the old secret key is erased. This allows honest users to verify that a signature was generated at certain time period.

The above guarantees are abstracted by an idealized functionality F_{kes} (cf. Figure 7). The main commands offered by F_{kes} are (i) (GEN), answered by (GEN, k_{kes}) for an (adversarially chosen) idealized public key k_{kes} , (ii) (SIGN, $k_{\text{kes}}, \mathbf{m}, j$), answered by (SIGN, σ) where $\mathbf{m}$ and σ are vectors of messages and signatures respectively, and (iii) (VFY, $k_{\text{kes}}, j, m, \sigma$), answered by (VFY, $m, j, \sigma, k_{\text{kes}}, f$) with $f \in \{0, 1\}$ indicating whether σ is a valid signature for message m at period j . For a realization of F_{kes} from a key evolving signature scheme we point to [21].

Functionality F_{kes}

F_{kes} is parameterized by the total number of updates T .

Variables: The functionality keeps track of the following variables, initialized to the values below:

1. **Keys** $:= \emptyset$: a set of tuples (P, v) , where P is a party and v is a public key.
2. **Sigs** $:= \emptyset$: a set of tuples (m, j, σ, v, f) where m is a message, j is a period number, σ is a signature, v is a public key, and f is a bit.
3. A period counter $\text{ctr}_P := 0$ for every party P .

Key generation: Upon receiving a message (GEN) from P , send (GEN, P) to the adversary. Upon receiving (GEN, P, v) from the adversary, verify that v does not appear in **Keys** or **Sigs**. Add (P, v) to **Keys**, set counter $\text{ctr}_P := 1$, and pass (GEN, v) to P .

Sign and Update. Upon receiving a message (SIGN, $v, j, \mathbf{m}$) from P :

1. If $(P, v) \notin \text{Keys}$, or $j < \text{ctr}_P$, or $j > T$, exit.
2. Send (SIGN, $v, j, \mathbf{m}$) to the adversary and obtain a value σ such that no entry $(\mathbf{m}[i], j, \sigma[i], v, 0)$ is recorded in **Sigs**, for all i . If it is, then output an error message to P and halt.
3. Add $(\mathbf{m}[i], j, \sigma[i], 1)$ to **Sigs** for all i .
4. Set $\text{ctr}_P := j + 1$.
5. Send (SIGN, σ) to P .

Signature Verification. Upon receiving a message (VFY, v', j, m, σ) from party P do:

1. If $v' = v$ and the entry $(m, j, \sigma, v, 1)$ is recorded in **Sigs**, then set $f := 1$. (This condition guarantees completeness: If the verification key v' is the registered one and σ is a legitimately generated signature for m , then the verification succeeds.)
2. Else, if $v' = v$, P is not corrupted, and no entry $(m, j, \sigma', v, 1)$ for any σ' is recorded in **Sigs**, then set $f := 0$ and record the entry $(m, j, \sigma, v, 0)$. (This condition guarantees unforgeability: If v' is the registered one, the signer is not corrupted, and never signed m , then the verification fails.)
3. Else, if there is an entry (m, j, σ, v', f') recorded in **Keys**, then let $f := f'$. (This condition guarantees consistency: All verification requests with identical parameters will result in the same answer.)
4. Else, if $j < \text{ctr}_P$, let $f := 0$ and record the entry $(m, j, \sigma, v, 0)$. Otherwise, if $j = \text{ctr}_P$, hand (VFY, m, j, σ, v') to the adversary. Upon receiving (VFY, m, j, ϕ) from the adversary let $f := \phi$ and record the entry (m, j, σ, v', ϕ) in **Sigs**. (This condition guarantees that the adversary is only able to forge signatures under keys belonging to corrupted parties for time periods corresponding to the current or future slots.)

Output (VFY, f) to P .

Fig. 7. Key-evolving signatures functionality F_{kes} .

E Stake-Based Voting

Leios requires a voting scheme that satisfies the following two conditions: (1) if all honest parties vote for a particular message, they are able to generate a certificate for that message; (2) if no honest party votes for a particular message, the adversary is unable to create such a certificate.

These guarantees are captured by a functionality $\mathcal{F}_{\text{sbv}}$, which is realized by a protocol π_{sbv} via stake-based voting, relying on a verifiable random function (VRF) and a key-evolving signature scheme (KES). The protocol π_{sbv} is secure against an adaptive adversary corrupting less than half of the total stake.

E.1 The SBV Functionality

The intuition behind the SBV functionality $\mathcal{F}_{\text{sbv}}$ (cf. Figure 8) is based on the standard way of modeling UC-secure signatures, where public keys and signa-

Functionality $\mathcal{F}_{\text{sbv}}$

Variables: The functionality keeps track of the following variables, initialized to the values below:

1. $\text{SD}^* := \perp$: stake distribution used.
2. $\text{Keys} := \emptyset$: a set of tuples (P, k) , where P is a party and k is a public key.
3. $\text{Votes} := \emptyset$: a set of tuples (k, j, m, v, h, f) where k is a public key, j is a period number, m is a message, v is a vote, and h and f are bits.
4. $\text{Certs} := \emptyset$: a set of tuples $(\mathbf{k}, j, m, c, f)$ where k is a public key, j is a period number, m is a message, v is a vote, and f is a bit.
5. A period counter $\text{ctr}_P := 0$ for every party P .

Key generation: Upon (GEN) from P : Pass (GEN, P) to the adversary and obtain a value k such that k does not appear in Keys , Votes , or Certs . Add (P, k) to keys and pass (GEN, v) to P .

Stake distribution: Upon $(\text{SETSD}, \text{SD})$ from P : If SD is invalid, exit. If $\text{SD}^* = \perp$, set $\text{SD}^* := \text{SD}$. If $\text{SD} \neq \text{SD}^*$, explode.

Voting: Upon $(\text{VOTE}, k, j, \mathbf{m})$ from P :

1. If $(P, k) \notin \text{Keys}$ or $j \leq \text{ctr}_P$, exit.
2. Pass $(\text{VOTE}, P, k, j, \mathbf{m})$ to the adversary and obtain a value $\mathbf{v}$ such that
 - (a) if $\mathbf{v} \neq \perp$, then $(k, j, \mathbf{m}[i], \mathbf{v}[i], \cdot, 0) \notin \text{Votes}$ for all i and
 - (b) if all honest parties except P have called VOTE for period j and only $\perp$ have been returned by the adversary so far, then $\mathbf{v} \neq \perp$.
3. If $\mathbf{v} \neq \perp$, add $(k, j, \mathbf{m}[i], \mathbf{v}[i], 1, 1)$ to Votes .
4. Set $\text{ctr}_P := j$.
5. Pass $(\text{VOTE}, \mathbf{v})$ to P .

Vote verification: Upon $(\text{VFYVOTE}, k, j, m, v)$ from P :

1. Pass $(\text{VFYVOTE}, k, j, m, v)$ to the adversary and obtain a bit ϕ .
2. Determine f as follows:
 - If $(k, j, m, v, \cdot, f') \in \text{Votes}$ for some f' , set $f := f'$.
 - Else if $(P, k) \in \text{Keys}$ and P is uncorrupted, set $f := 0$.
 - Else, set $f := \phi$ and add $(k, j, m, v, 0, \phi)$ to Votes .
3. Pass $(\text{VFYVOTE}, f)$ to P .

Certificate generation: Upon $(\text{GENCERT}, \mathbf{k}, j, m, \mathbf{v})$ from P :

1. Run the verification in VFYVOTE with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i ; if not all of them succeed, exit.
2. Pass $(\text{GENCERT}, \mathbf{k}, j, m, \mathbf{v})$ to the adversary and obtain a value c such that
 - (a) if $c \neq \perp$, then $(\mathbf{k}, j, m, c, 0) \notin \text{Certs}$ and
 - (b) if $\mathbf{k}$ includes all keys k for which $(k, j, m, v, 1, 1) \in \text{Votes}$ and all honest parties have voted for m period j , then $c \neq \perp$.
3. If $c \neq \perp$, add $(\mathbf{k}, j, m, c, 1)$ to Certs .
4. Pass $(\text{GENCERT}, c)$ to P .

Certificate verification: Upon $(\text{VFYCERT}, j, m, c)$ from P :

1. Pass $(\text{VFYCERT}, j, m, c)$ to the adversary and obtain a bit ϕ .
2. Determine f as follows:
 - If $(j, m, c, f') \in \text{Certs}$ for some f' , set $f := f'$.
 - Else if $(\cdot, j, m, \cdot, 1, 1) \notin \text{Votes}$, set $f := 0$.
 - Else, set $f := \phi$ and add (k, j, m, c, ϕ) to Votes .
3. Pass $(\text{VFYCERT}, f)$ to P .

Fig. 8. Stake-based voting functionality $\mathcal{F}_{\text{sbv}}$.

tures are “symbolic” strings chosen by the adversary, and unforgeability is enforced based on what was signed by uncorrupted parties.

Functionality $\mathcal{F}_{\text{sbv}}$ proceeds in periods, where parties can only sign (vectors of) messages under strictly increasing period numbers. This is due to the realization based on KESs, required to obtain adaptive security (see Appendix D). To that end $\mathcal{F}_{\text{sbv}}$ keeps counters ctr_P that remember, on a per-party basis, the latest period for which a message was signed.

The functionality keeps track of

- public keys using a set Keys of pairs (P, k) , meaning that public key k belongs to party P ;

- votes using a set **Votes** consisting of tuples (k, j, m, v, h, f) , meaning that v is a period- j vote for m under public key k ; $f \in \{0, 1\}$ records whether v is valid and h whether the voter was honest at the time of creation; and
- certificates using a set **Certs** consisting of tuples $(\mathbf{k}, j, m, c, f)$, meaning that c is a period- j certificate for message m created using votes under keys $\mathbf{k}$; $f \in \{0, 1\}$ records whether c is valid.

Key generation. A party requests a public key from $\mathcal{F}_{\text{sbv}}$ using the GEN command. $\mathcal{F}_{\text{sbv}}$ then asks the adversary to provide a symbolic key string that is different from any keys appearing in sets **Keys**, **Votes**, and **Certs**.

Stake distribution. In order to realize $\mathcal{F}_{\text{sbv}}$ using stake-based voting, after generating their keys, parties must provide a valid stake distribution $\text{SD} = \{(P, s_P, k_P)\}_P$ to $\mathcal{F}_{\text{sbv}}$, using command SETSD. SD is valid if it satisfies $\sum_P s_P = 1$ and k_P is a key belonging to P for all P . Crucially, parties must reach agreement on the stake distribution externally, as the functionality only works if all honest parties input the same SD (otherwise, the adversary is given full control over $\mathcal{F}_{\text{sbv}}$).

Vote generation and verification. Using command VOTE, parties may obtain individual votes for arbitrary messages in a vector $\mathbf{m}$ for a particular period, i.e., all votes for a given period must be requested simultaneously. $\mathcal{F}_{\text{sbv}}$ then requests a vector $\mathbf{v}$ of symbolic vote strings from the adversary, who may choose to also reply with the special symbol $\perp$. This flexibility allows to realize $\mathcal{F}_{\text{sbv}}$ via sortition. However, in order to be useful, the functionality will enforce that at least one honest party is given a non- $\perp$ vote in each period, which will be sufficient for the generation of a certificate. $\mathcal{F}_{\text{sbv}}$ ensures that if non- $\perp$ votes are returned by the adversary, they do not already appear in **Votes** as invalid.

Vote verification is akin to standard signature verification in UC, where the adversary is asked for a bit ϕ , determining vote validity, unless

- the vote to be verified is already known to be valid or invalid, in which case the answer (the f -bit) in **Votes** is used, or
- the key under which the signature is to be verified belongs to an honest party, in which case the answer is always 0 (in order to ensure vote unforgeability).

Certificate generation and verification. Parties may request certificates for messages m by providing a set of corresponding votes under particular keys. The adversary is again asked to provide a symbolic certificate string. It may also return $\perp$, except when all honest parties have voted for m in the period in question and the set of votes includes all honestly generated votes for m . $\mathcal{F}_{\text{sbv}}$ ensures that if a non- $\perp$ certificate is returned by the adversary, it does not already appear in **Certs** as invalid.

Certificate verification follows the same pattern as above, where the adversary is asked for a bit ϕ , determining certificate validity, unless

- the certificate to be verified is already known to be valid or invalid, in which case the answer (the f -bit) in **Certs** is used, or

Protocol π_{sbv}

Variables: The protocol keeps track of the following variables, initialized to the values below:

1. $\text{SD}^* := \perp$: stake distribution used.
2. $\text{Keys} := \emptyset$: the set of public keys of the party.
3. A period counter ctr .

Key generation: Upon (GEN) from P : Pass (GETKEY) to F_{vrf} and (GEN) to F_{kes} and obtain keys k_{vrf} and k_{kes} , respectively. Add $(k_{\text{vrf}}, k_{\text{kes}})$ to Keys and pass (GEN, $(k_{\text{vrf}}, k_{\text{kes}})$) to P .

Stake distribution: Upon (SETSD, SD) from P : If $\text{SD}^* = \perp$ and SD is valid, set $\text{SD}^* := \text{SD}$.

Voting: Upon (VOTE, $(k_{\text{vrf}}, k_{\text{kes}}), j, \mathbf{m}$) from P :

1. If $(k_{\text{vrf}}, k_{\text{kes}}) \notin \text{Keys}$ or $j \leq \text{ctr}$, exit.
2. Pass (EVAL, k_{vrf}, j) to F_{vrf} and obtain (EVAL, y, π), where y is the VRF output and π is the VRF proof.
3. If $\text{voteCount}(\text{SD}^*, k_{\text{vrf}}, y) = 0$, pass (VOTE, $\perp$) to P and exit.
4. Pass (SIGN, $k_{\text{kes}}, j, \mathbf{m}'$) to F_{kes} , where $\mathbf{m}'[i] = (j, \mathbf{m}[i])$, and obtain (SIGN, $\mathbf{s}$), where $\mathbf{s}$ is a vector of signatures.
5. Let $\mathbf{v}$ be the vector of votes $\mathbf{v}[i] := (y, \pi, \mathbf{s}[i])$ for all i .
6. Set $\text{ctr} := j$.
7. Pass (VOTE, $\mathbf{v}$) to P .

Vote verification: Upon

(VFYVOTE, $(k_{\text{vrf}}, k_{\text{kes}}), j, m, (y, \pi, \sigma)$) from P :

1. Pass (VERIFY, $k_{\text{vrf}}, j, y, \pi$) to F_{vrf} and obtain (VERIFY, f_{vrf}), where f_{vrf} is a bit.
2. Set $f_{\text{cnt}} := 1$ if $\text{voteCount}(\text{SD}^*, k_{\text{vrf}}, y) > 0$ and $f_{\text{cnt}} := 0$ otherwise.
3. Pass (VFY, $k_{\text{kes}}, j, m, \sigma$) to F_{kes} and obtain (VFY, f_{kes}), where f_{kes} is a bit.
4. Pass (VFYVOTE, $f_{\text{vrf}} \wedge f_{\text{cnt}} \wedge f_{\text{kes}}$) to P .

Certificate generation: Upon

(GENCERT, $\mathbf{k}, j, m, \mathbf{v}$) from P :

1. Run the verification in VFYVOTE with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i ; if not all of them succeed, exit.
2. If $\sum_i \text{voteCount}(\text{SD}^*, \mathbf{k}[i].k_{\text{vrf}}, \mathbf{v}[i].y) < T$, exit.
3. Pass (GENCERT, $(\mathbf{k}, \mathbf{v})$) to P .

Certificate verification: Upon

(VFYCERT, $j, m, (\mathbf{k}, \mathbf{v})$) from $\mathcal{F}_{\text{sbv}}$:

1. Run the verification in VFYVOTE with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i ; set $f_{\text{vfy}} := 1$ if all of them succeed and $f_{\text{vfy}} := 0$ otherwise.
2. Set $f_{\text{cnt}} := 1$ if $\sum_i \text{voteCount}(\text{SD}^*, \mathbf{k}[i].k_{\text{vrf}}, \mathbf{v}[i].y) \geq T$ and $f_{\text{cnt}} := 0$ otherwise.
3. Pass $f_{\text{vfy}} \wedge f_{\text{cnt}}$ to $\mathcal{F}_{\text{sbv}}$.

Fig. 9. Protocol π_{sbv} to realize $\mathcal{F}_{\text{sbv}}$.

- no honest party has voted for the message in question in the period in question, in which case the answer is always 0 (in order to ensure vote unforgeability).

E.2 The SBV Protocol

For completeness, this work provides a protocol π_{sbv} (cf. Figure 9) that realizes $\mathcal{F}_{\text{sbv}}$. The protocol is based on sortition, i.e., in each period, a representative subset of the parties is elected in a VRF-based lottery and casts a vote. The certification generation is trivial, i.e., the votes are simply concatenated. This is sufficient for Leios, but more elaborate certification, in which votes are suitably aggregated, could be used to obtain more efficient realizations of $\mathcal{F}_{\text{sbv}}$. The protocol is described in the $\{F_{\text{vrf}}, F_{\text{kes}}\}$ -hybrid model.

Sortition. In the following it is assumed for simplicity that the stake distribution $\{s_P\}_P$ can be approximated by a set of weights w_P such that $s_P = w_P/W$ for $W = \sum_P w_P$. The sortition simply uses the output y of a VRF as randomness to sample a number of votes for party P from a binomial distribution $\text{Bin}(w_P, p)$

for some parameter p used to calibrate the expected total number Wp of votes. For a stake distribution SD and the key k_{vrf} belonging to P , the number of votes is denoted by $\text{voteCount}(\text{SD}, k_{\text{vrf}}, y)$.

Key generation. The public keys for π_{sbv} simply consist of public keys k_{vrf} and k_{kes} obtained from F_{vrf} and F_{kes} , respectively.

Stake distribution. The party simply remembers the stake distribution input via command SETSD (if it is valid).

Vote generation and verification. To create votes for messages in a vector $\mathbf{m}$ in period j , a party first evaluates the VRF on j to obtain (y, π) , where y is the VRF output and π is the proof. Then, it computes $\text{voteCount}(\text{SD}^*, k_{\text{vrf}}, y)$. If the result is 0, the party was not elected to vote in period j and the output is $\perp$. Otherwise, the party signs $(j, \mathbf{m}[i])$ for all i with the KES scheme, obtaining a vector of signatures σ , and produces the votes $\mathbf{v}[i] = (y, \pi, \sigma[i])$.

Votes are verified by checking the VRF value y , the KES signature, and that voteCount on y is non-zero.

Certificate generation and verification. Certificates are generated by concatenation from any vector $\mathbf{v}$ of valid votes belonging to keys $\mathbf{k}$ with combined voteCount at least threshold T for the period in question. Given a fraction $\alpha < 1/2$ of relative adversarial stake, note that with high probability, the adversary will get a voteCount of approximately αWp whereas the honest parties will get approximately $(1 - \alpha)Wp$.

Certificate verification proceeds by verifying the individual votes and ensuring the voteCount is at least T .

E.3 Security

Environment. Protocol π_{sbv} securely realizes $\mathcal{F}_{\text{sbv}}$ against an adaptive adversary corrupting at most half of the stake.

Theorem 7. *Let $0 \leq \alpha < 1/2$ be a constant and $0 < \delta < 1 - 2\alpha$. Further, let $W \in \mathbb{N}$ and $p \in (0, 1]$ as above. Then, π_{sbv} with threshold $T := (1 - \delta)(1 - \alpha)Wp$ securely UC-realizes $\mathcal{F}_{\text{sbv}}$ against an adaptive adversary corrupting at most half of the stake, except with error $e^{-\Omega(\delta^2 \alpha Wp)}$.*

Proof. The simulator for protocol $\mathcal{S}_{\text{sbv}}$ is provided in Figure 10. It simulates keys, signatures, and certificates in a straightforward way.

To prove indistinguishability of the real and ideal worlds, it is only necessary to show that the “overrides” in $\mathcal{F}_{\text{sbv}}$, where answers by the adversary are overwritten, only occur with small probability with $\mathcal{S}_{\text{sbv}}$.

These overrides are:

Protocol $\mathcal{S}_{\text{sbv}}$

Variables: The simulator keeps track of the following variables, initialized to the values below:

1. $\text{SD}^* := \perp$: stake distribution used.
2. $\text{Keys} := \emptyset$: a set of tuples $(P, (k_{\text{vrf}}, k_{\text{kes}}))$, where P is a party and $(k_{\text{vrf}}, k_{\text{kes}})$ is a public key.

The simulator also internally runs copies of F_{vrf} and F_{kes} as well as of an adversary $\mathcal{A}$, connected to F_{vrf} and F_{kes} .

Key generation: Upon (GEN, P) from $\mathcal{F}_{\text{sbv}}$: Pass, on behalf of P , (GETKEY) to F_{vrf} and (GEN) to F_{kes} and obtain keys k_{vrf} and k_{kes} , respectively. Add $(P, (k_{\text{vrf}}, k_{\text{kes}}))$ to Keys and pass $(k_{\text{vrf}}, k_{\text{kes}})$ to $\mathcal{F}_{\text{sbv}}$.

Stake distribution: Upon $(\text{SETSD}, \text{SD})$ from P : If $\text{SD}^* = \perp$, set $\text{SD}^* := \text{SD}$. If $\text{SD} \neq \text{SD}^*$, simulate real world exactly for the remainder of the execution.

Voting: Upon $(\text{VOTE}, P, (k_{\text{vrf}}, k_{\text{kes}}), j, \mathbf{m})$ from $\mathcal{F}_{\text{sbv}}$:

1. Pass, on behalf of P , $(\text{EVAL}, k_{\text{vrf}}, j)$ to F_{vrf} and obtain (EVAL, y, π) , where y is the VRF output and π is the VRF proof.
2. If $\text{voteCount}(\text{SD}^*, k_{\text{vrf}}, y) = 0$, pass $\perp$ to $\mathcal{F}_{\text{sbv}}$ and exit.
3. Pass, on behalf of P , $(\text{SIGN}, k_{\text{kes}}, j, \mathbf{m}')$ to F_{kes} , where $\mathbf{m}'[i] = (j, \mathbf{m}[i])$, and obtain $(\text{SIGN}, \mathbf{s})$, where $\mathbf{s}$ is a vector of signatures.

4. Let $\mathbf{v}$ be the vector of votes
 $\mathbf{v}[i] := (y, \pi, \mathbf{s}[i])$ for all i .
5. Pass $(\text{VOTE}, \mathbf{v})$ to $\mathcal{F}_{\text{sbv}}$.

Vote verification: Upon $(\text{VFYVOTE}, (k_{\text{vrf}}, k_{\text{kes}}), j, m, (y, \pi, \sigma))$ from $\mathcal{F}_{\text{sbv}}$:

1. Pass $(\text{VERIFY}, k_{\text{vrf}}, j, y, \pi)$ to F_{vrf} and obtain $(\text{VERIFY}, f_{\text{vrf}})$, where f_{vrf} is a bit.
2. Set $f_{\text{cnt}} := 1$ if $\text{voteCount}(\text{SD}^*, k_{\text{vrf}}, y) > 0$ and $f_{\text{cnt}} := 0$ otherwise.
3. Pass $(\text{VFY}, k_{\text{kes}}, (j, m), \sigma, \pi)$ to F_{kes} and obtain $(\text{VFY}, f_{\text{kes}})$, where f_{kes} is a bit.
4. Pass $f_{\text{vrf}} \wedge f_{\text{cnt}} \wedge f_{\text{kes}}$ to $\mathcal{F}_{\text{sbv}}$.

Certificate generation: Upon $(\text{GENCERT}, \mathbf{k}, j, m, \mathbf{v})$ from $\mathcal{F}_{\text{sbv}}$:

1. If $\sum_i \text{voteCount}(\text{SD}^*, \mathbf{k}[i].k_{\text{vrf}}, \mathbf{v}[i].y) < T$, pass $\perp$ to $\mathcal{F}_{\text{sbv}}$ and exit.
2. Pass $(\mathbf{k}, \mathbf{v})$ to $\mathcal{F}_{\text{sbv}}$.

Certificate verification: Upon $(\text{VFYCERT}, j, m, (\mathbf{k}, \mathbf{v}))$ from $\mathcal{F}_{\text{sbv}}$:

1. Run the verification in VFYVOTE with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i ; set $f_{\text{vfy}} := 1$ if all of them succeed and $f_{\text{vfy}} := 0$ otherwise.
2. Set $f_{\text{cnt}} := 1$ if $\sum_i \text{voteCount}(\text{SD}^*, \mathbf{k}[i].k_{\text{vrf}}, \mathbf{v}[i].y) \geq T$ and $f_{\text{cnt}} := 0$ otherwise.
3. Pass $(\text{VFYVOTE}, f_{\text{vfy}} \wedge f_{\text{cnt}})$ to P .

Fig. 10. Simulator for protocol π_{sbv} .

1. Forcing the output of a vote when the adversary wants to output $\perp$ in VOTE . This only happens if voteCount is 0 for all honest parties, which happens with probability at most $(1 - p)^{(1-\alpha)W}$.
2. Forcing the output of a certificate when the adversary wants to output $\perp$ in GenCert . This only happens when the honest parties do not get a combined vote count of at least $(1 - \delta)(1 - \alpha)Wp$, which happens with probability at most $e^{-\Omega(\delta^2(1-\alpha)Wp)}$ by a simple Chernoff bound.
3. Forcing the output of $f = 0$ when the adversary has enough votes to cross the threshold $T = (1 - \delta)(1 - \alpha)Wp$ without honest votes. Using the fact that $\delta < 1 - 2\alpha$ —and therefore $(1 + \delta)\alpha < (1 - \delta)(1 - \alpha)$ —this only happens with probability at most $e^{-\Omega(\delta^2\alpha Wp)}$ by a simple Chernoff bound.

The theorem follows. $\square$

F Simple Leios: Analysis

This section contains the deferred security analysis of Simplified Leios.

The main part of the analysis is to give an estimate on the probability that a pipeline is “successful,” as defined formally below. Furthermore, the analysis bounds the liveness delay incurred by Simplified Leios. This section assumes that there is full (honest) participation. It is easy to see that under low participation, Leios inherits safety and liveness from the base protocol.

Successful pipelines. A successful pipeline produces at least one VOTE2-certified EB that covers all PROPOSE-stage IBs:

Definition 5. A pipeline instance *is* successful *if*

1. *its PROPOSE stage produces at least $\alpha_H(1 - \varepsilon)Lf_I$ honest IBs,*²² *and*
2. *its ENDORSE stage produces an honest EB that gets VOTE2-certified by the end of the VOTE2 stage, and*
3. *by the end of the VOTE2 stage, all honest input blocks from the PROPOSE stage are (indirectly) referenced by all VOTE2-certified EB from the ENDORSE stage.*

Recall that the protocol execution is divided into slices of length L . In order for a slice to be “useful,” the number of IBs generated during it must be in a reasonable interval, and an honest majority of the EBs generated must be from honest parties. This is captured by the following definition:

Definition 6. Let $\varepsilon > 0$. A slice *is* ε -successful *if it produces*

- (1) *at least $\alpha_H(1 - \varepsilon)Lf_I$ honest IBs,*
- (2) *no more than $(1 + \varepsilon)Lf_I$ IBs, and*
- (3) *an honest majority of endorser blocks.*

The next step is to upper bound the probability of an unsuccessful slice as stated in Lemma 5.

Lemma 5. Let ε such that $0 < \varepsilon < 2\alpha_H - 1$. A slice *is* ε -successful *except with probability*

$$\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) := e^{-\varepsilon^2 \Omega(f_I L)} + e^{-\varepsilon^2 \Omega(f_E L)} .$$

Proof. The upper bound follows from a union bound over the three properties of a successful slice and the Chernoff-Hoeffding bound as given in Appendix B.

1. Let $X_{ij} \in \{0, 1\}$ be the indicator random variable that honest party j gets the right to produce an IB in slot i (aka slot leadership), let there be n_H honest parties, where the honest party with index $j \in [1, n_H]$ holds relative stake α_j , and let

$$X = \sum_{i=1}^L \sum_{j=1}^{n_H} X_{ij}$$

²²Recall that α_H denotes the fraction of honest stake.

be the random variable representing the number of honest slot leaderships during the given slice. Since $\Pr[X_{ij}] = \alpha_j f_I$,

$$E[X] = \alpha_H f_I L ,$$

and, by the Chernoff-Hoeffding bound from Appendix B, the probability that Property (1) is violated is thus

$$P_1 \leq \exp\left(-\frac{\varepsilon^2 \alpha_H f_I L}{2}\right).$$

2. In expectation, $f_I L$ IBs are generated during the slice. By the Chernoff-Hoeffding bound, along the lines of the above, the probability that Property (2) is violated is thus

$$P_2 \leq \exp\left(-\frac{\varepsilon^2 f_I L}{3}\right).$$

3. In expectation, a slice produces $\alpha_H f_E L$ honest endorser blocks, and at most $(1 - \alpha_H) f_E L$ adversarial ones. The probability that there are at least as many adversarial as honest endorser blocks can thus be estimated by a union bound over the probabilities that the honest production undercuts its expectation by a factor of $(1 - \varepsilon)$ and the adversarial production exceeds its expectancy by a factor of $(1 + \varepsilon)$, as long as $(1 - \varepsilon)\alpha_H > (1 + \varepsilon)(1 - \alpha_H)$, which is equivalent to $\varepsilon < 2\alpha_H - 1$. Thus,

$$P_3 \leq \exp\left(-\frac{\varepsilon^2 \alpha_H f_E L}{2}\right) + \exp\left(-\frac{\varepsilon^2 (1 - \alpha_H) f_E L}{3}\right).$$

□

Before proceeding, recall that F_{diff} (see Section 3) guarantees the delivery of an IB within $\beta + k \cdot \tau$ time. Here, β represents the base delay for transmitting a message through the network (assuming no queuing), and $k \cdot \tau$ accounts for the maximum queuing delay, where τ is the time required to download the IB over a single communication link, and k is the number of fresher IBs in transit.

In order to analyze the success of an entire pipeline instance, consider the following parameter settings (cf. Section 4.1 for intuition):

$$\rho := \frac{\beta}{L}, \quad f_I := \frac{\lambda - \rho}{(1 + \varepsilon)(\lambda + 1)} \cdot \frac{1}{\tau}, \quad \text{and} \quad \mu := \frac{\lambda^2 + \lambda - \rho}{1 + \rho}.$$

The following lemma “reduces” the notion of pipeline success to the successfulness of slices.

Lemma 6. *For any $\varepsilon > 0$, a pipeline instance is successful if all its slices are ε -successful.*

Proof. It is sufficient to demonstrate the following facts:

1. all honest IBs are delivered by the end of the DELIVER1 stage; and
2. all IBs seen by an honest party by the end of the DELIVER1 stage are seen by all honest parties by the end of the DELIVER2 stage.

Success of the LINK stage then guarantees that all honest IBs from the PROPOSE stage are referenced by all honest VOTE1-certified EBs from the LINK stage, and success of the ENDORSE stage guarantees that an (honest) VOTE2-certified EB is produced that covers a majority of all VOTE1-certified EBs of the LINK stage. The latter implies that at least one honest LINK-stage EB is referenced (as there is a majority of honest parties).

Fact (1) is proved by ensuring that the IB production during the first $\lambda + 1$ pipeline slices essentially “fits” into the λL slots of the DELIVER1 stage. Specifically, consider any honest IB from the PROPOSE stage. As per the guarantees of F_{diff} , the IB is globally delivered

$$\beta + k \cdot \tau \text{ slots after sending}$$

if at that point there are at most k IBs with newer slot numbers in the network.²³ Since all slices in the pipeline are assumed to be ε -successful, there are at most $(1 + \varepsilon)(\lambda + 1)Lf_1$ IBs produced during all of PROPOSE and DELIVER1. Therefore, global delivery is guaranteed if

$$\beta + ((1 + \varepsilon)(\lambda + 1)Lf_1)\tau \stackrel{!}{\leq} \lambda L ,$$

This is satisfied if

$$f_1 \leq \frac{\lambda - \rho}{(1 + \varepsilon)(\lambda + 1)} \cdot \frac{1}{\tau} ,$$

which corresponds to the choice of f_1 above.

One similarly argues that Fact (2) is true if the IB production during the first $\lambda + \mu + 2$ slices fits into the $\mu + 1$ slices corresponding to the LINK and DELIVER2 stages. This yields the condition

$$\beta + ((1 + \varepsilon)(\lambda + \mu + 2)Lf_1)\tau \stackrel{!}{\leq} (\mu + 1)L ,$$

which is satisfied if

$$f_1 \leq \frac{(\mu + 1) - \rho}{(1 + \varepsilon)(\lambda + \mu + 2)} \cdot \frac{1}{\tau} .$$

Given the choice of f_1 , this is satisfied if μ is chosen such that

$$\frac{\lambda - \rho}{\lambda + 1} \leq \frac{(\mu + 1) - \rho}{\lambda + \mu + 2} ,$$

which requires

$$\mu \geq \frac{\lambda^2 + \lambda - \rho - 1}{\rho + 1} .$$

This is satisfied by the choice of μ above. □

²³Recall that τ is the time it takes to send a body over a single “hop” in the network.

Recall from Section 4.1 that the adversary can delay the availability of adversarial IBs referenced (indirectly) by VOTE2-certified EBs. The following lemma quantifies this delay.

Lemma 7. *Any IB (indirectly) referenced in a VOTE2-certified EB from slot s is downloaded by every honest party no later than in slot $s + \nu L$ for*

$$\nu := \frac{(\lambda - \rho)(\lambda + \mu + 2) + (\lambda + 1)\rho}{\rho + 1} = O(\lambda^3) .$$

Proof. Along the lines of the argumentation in the proof of Lemma 6, we demand that input block data available at the onset of the VOTE1 to any honest party will be downloaded by every honest party within the next ν slices.

In particular, we demand that

$$\beta + ((1 + \varepsilon)f_I(2 + \lambda + \mu + \nu)L)\tau \stackrel{!}{\leq} \nu L , \quad (1)$$

This is satisfied if

$$f_I \leq \frac{\nu - \rho}{(1 + \varepsilon)(\lambda + \mu + \nu + 2)} \cdot \frac{1}{\tau} .$$

Given the choice of f_I , this is satisfied if ν is chosen such that

$$\frac{\lambda - \rho}{\lambda + 1} \leq \frac{\nu - \rho}{\lambda + \mu + \nu + 2} ,$$

which requires

$$\nu \geq \frac{(\lambda - \rho)(\lambda + \mu + 2) + (\lambda + 1)\rho}{\rho + 1} ,$$

which corresponds to the choice of ν . □

Main theorems. In the following, let t be the time the protocol is run, $\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$ the probability a slice fails (cf. Lemma 5), and $\varepsilon_{\text{vote}}(t)$ the error probability of the voting scheme if run for time t . Further, assume the base protocol satisfies persistence with parameter w and ledger quality η with liveness u ; let $\varepsilon_{\text{base}}(t)$ be the error of the base protocol when run for time t .

Theorem 1. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Then, Simplified Leios achieves persistence with parameter w' and liveness with parameters r' and u' , where $w' = w$, $r' = \eta + (6 + \lambda + \mu)L$, and $u' = u + (6 + \lambda + \mu + \nu)L$, except with probability $O(t)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) + \varepsilon_{\text{vote}}(t) + \varepsilon_{\text{base}}(t)$.*

Proof. The only way for persistence to be violated is if the adversary manages to forge an EB certificate for an EB that references IBs that are not available. Thus, under the assumption that there are no forgeries in the voting scheme, the overlay protocol inherits persistence from the base protocol.

As for liveness, note that each slice is ε -successful except with probability $\mathcal{O}(t)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$, which follows from Lemma 5 and a union bound. To determine r' , consider any interval of length η (the quality parameter of the base protocol), producing an honest base-protocol block (with some timestamp s). This block will (except for error $\mathcal{O}(t)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$) reference a successful pipeline that started at time $s - d$ for $d \in [(5 + \lambda + \mu)L, (6 + \lambda + \mu)L]$, accounting for one pipeline length plus the fact that the transaction may be issued out of sync with the pipelines (by up to one slice length L). As a consequence, any time interval $[t_0, t_0 + \eta]$ will produce a base-protocol block that references an honest IB from interval $[t_0 - (6 + \lambda + \mu)L, t_1]$. Therefore, a transaction tx will be referenced in a base protocol block, if the honest parties receive it via FTCHPLD during the r' slots.

As for u' , any IB or transaction to be settled takes the extra detour via the pipeline (of duration $(5 + \lambda + \mu)L$) plus possible synchronization with the pipelines of duration L . Such an IB is thus represented in the base chain after time $(6 + \lambda + \mu)L + u$ —although possibly not yet downloaded by value by all honest parties. By Lemma 7, those IBs will be fully available νL after the DELIVER2 stage, yielding the given u' . $\square$

Obviously, the guarantee given by Theorem 1 is quite weak as it is only meaningful for large enough L due to the union bound over t . Indeed, if the failure probability of the pipelines is too large, then, depending on the underlying (black-box) protocol, it remains unclear with what probability honest base-protocol payloads can be guaranteed to include an EB from a successful pipeline—an uncertainty that is eliminated in Full Leios.

Theorem 2. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Simplified Leios achieves $(\varepsilon_{\text{tp}}(t), R)$ -throughput for*

$$\varepsilon_{\text{tp}}(t) = \mathcal{O}(t)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) + \varepsilon_{\text{vote}}(t) + \varepsilon_{\text{base}}(t)$$

and

$$R = \left(\frac{1 - \varepsilon}{1 + \varepsilon} \frac{\lambda - \rho}{\lambda + 1} \right) \min(L/\eta, 1) \cdot \alpha_H \cdot \mathcal{C}.$$

Before we proceed to the proof, recall that due to their small size compared to IBs, EBs and votes are modeled as consisting of headers only when it comes to diffusing them via F_{diff} , and that the bandwidth needed for the base protocol is modeled separately (as hybrid functionalities assumed by the base protocol). In practice, this means that some of the overall bandwidth $\mathcal{C}$ needs to be allocated to the base protocol and to the diffusion of EBs and votes. However, observe that the larger $\mathcal{C}$ is, the smaller the fraction of bandwidth taken up by those parts becomes as the amount of traffic produced by them does not increase as one cranks up the IB data rate (by increasing the size of IBs).

Specifically, let ℓ be the bandwidth needed for the base protocol as well as the EBs and votes, which leaves capacity $\bar{\mathcal{C}} = \mathcal{C} - \ell$ that can be allocated to the

diffusion of IBs. Then, the throughput achieved by simplified Leios is in fact

$$\bar{R} = \left(\frac{1 - \varepsilon \lambda - \rho}{1 + \varepsilon \lambda + 1} \right) \min(L/\eta, 1) \alpha_H \bar{C}.$$

This can be written as $\bar{R} = (1 - \delta)R$ for $\delta = \ell/\bar{C}$, and hence, for large enough total capacity $\bar{C}$, the throughput of Simplified Leios is still arbitrarily close to R . Keeping this observation in mind, for ease of exposition, we can thus ignore the effect of non-IB messages in the following proof, and assume that the full network capacity is allocated to IBs.

Proof (of Theorem 2). The rate at which honest IBs are produced in an ε -successful slice is at least

$$r = (1 - \varepsilon) \alpha_H f_I |\mathbf{B}| = \frac{1 - \varepsilon \lambda - \rho}{1 + \varepsilon \lambda + 1} \alpha_H \frac{|\mathbf{B}|}{\tau} = \frac{1 - \varepsilon \lambda - \rho}{1 + \varepsilon \lambda + 1} \alpha_H \bar{C},$$

where the first equality follows from the choice of f_I (and the definition of ε -successfulness), and the second one is due to the fact that $\tau = |\mathbf{B}|/\bar{C}$.

Over an execution of time t , except with probability $\varepsilon_{\text{tp}}(t)$ as declared by the theorem, all slices are successful, and thus, all EBs that settle reference a successful pipeline (Lemma 6). Thus, declaring the above error probability (depending on t_0 which we still need to determine), we now assume an execution over time t where all pipelines that terminate within time t are successful.

The duration of a single pipeline instance involves $t_P = (\mu + \lambda + \nu + 5)L$ slots (including late delivery of the actual payloads after νL slots), and k instances involve $((\mu + \lambda + \nu + 5) + (k - 1))L$ slots.

As already observed in the proof of Theorem 1, the IBs included in a VOTE2-certified EB that is to settle in the base protocol, do settle within $(t_P + 1)L + u$ after their creation (where u is the liveness parameter of the base protocol).

By the assumption of base-ledger quality η , the base protocol settles a payload once per η slots, and thus, during a duration of

$$t = (t_P + k)L + u + \eta$$

settles the honest (and unique) IB payloads of at least $k \cdot \min(L/\eta, 1)$ (successful) pipelines, where $u + \eta$ accounts for the losses due to base-ledger liveness u and possibly one “partial” quality period. The protocol thus settles data at rate

$$r' = \frac{kL}{t} \cdot r \cdot \min(L/\eta, 1) = \frac{kL}{t} \cdot R = \frac{kL}{(t_P + k)L + u + \eta} \cdot R.$$

Given any $\gamma > 0$ in Definition 4, we can thus choose t_0 such that

$$(1 - \gamma) \geq \frac{t_0 - t_P L - u - \eta}{t_0},$$

yielding, for $t \geq t_0$, that

$$\frac{|L_{\text{san}}^{(t)}|}{t} \geq (1 - \gamma)R,$$

as claimed. Note that, considering u and η as constants, this choice implies that $t_0 = \mathcal{O}(t_P L/\gamma) = \mathcal{O}(\lambda^3 L/\gamma)$. $\square$

G Full Leios: Analysis

This section contains the deferred security analysis of Full Leios.

It is easy to see that the guarantees of Simple Leios remain intact for Full Leios. We now state the additional guarantees.

Lemma 8. *Given a sequence of adjacent pipelines, the expected fraction of unsuccessful pipelines is at most $\varphi_{\text{fail}} = \mathcal{O}(\lambda^2)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$.*

Proof. Slice failures are independent and occur with probability $\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$. A slice failure can only affect $\mathcal{O}(\lambda^2)$ pipelines, and the lemma follows by a simple union bound. $\square$

In particular, note that this rate vanishes exponentially fast in L .

Lemma 9. *All successful pipelines are covered from the base protocol, except with probability $\varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$.*

Proof. We first argue that an honest EB of a successful pipeline is eligible for base-protocol inclusion, i.e, it is VOTE2-certified. Now, consider an honest EB candidate for VOTE2-stage voting by an honest party P . Any previous pipeline that P requires to be referenced by EB (as defined by the new voting rule) will indeed be referenced by EB as the respective certificate from P 's view was available to the producer of EB by the end of that pipeline. Thus every honest EB will be VOTE2-certified and will be thus be eligible for base-protocol inclusion.

We now show that all successful pipelines will eventually be referenced in the ledger. For this, consider any successful pipeline instance i ending at time slot t_i (and producing an honest VOTE2-certified EB_i), and assume that an EB of that pipeline does not get *directly* included as a ledger payload (otherwise, we are done).

Furthermore, consider the point in time $t \geq t_i + L$ when the next honest payload p enters the ledger. Ledger quality guarantees that $t \leq t_i + \eta$. We distinguish two cases:

1. p contains an EB_j from a pipeline instance $j > i + 1$, or,
2. p contains an EB_j from pipeline instance $j = i + 1$.

If $j > i + 1$ then EB_j references EB_i , as the recursion spans 2η . For the case $j = i + 1$, consider the next honest payload p' that enters the ledger after p , which, due to ledger quality, happens during time interval $[t, t + \eta] \subseteq [t_i, t_i + 2\eta]$. Since p' neither directly includes EB_i nor EB_j , it directly includes an EB from a pipeline instance $k \in [i + 2, i + \lceil 2\eta/L \rceil]$, indirectly covering EB_i because of EB recursion span 2η . $\square$

Theorem 3. *Full Leios achieves persistence with parameter $w' = w$, and, on average (over all transactions), the protocol achieves liveness with parameters $r' = L + \frac{L}{1 - \varepsilon_{\text{fail}}}$, and $u' = \frac{L}{1 - \varepsilon_{\text{fail}}} + \mathcal{O}(\lambda^3)L + \eta + u$, except with probability $\varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$.*

Proof. Persistence (as a property of the base protocol) remains intact: $w' = w$. Average liveness follows from the following facts:

- the base protocol satisfies liveness with parameter u ,
- the expected wait time for inclusion in a successful pipeline is (at most) $L + \frac{L}{1 - \varepsilon_{\text{fail}}(L)}$ (the extra L accounts for the fact that the initial execution of FTCHPLD may be out of sync with the pipeline slices),
- the maximal time delay for the pipeline delivery of all IBs is $\mathcal{O}(\lambda^3)L$,
- a VOTE2-certified EB guaranteed to reference the transaction in question needs to wait for at most two new honest payloads entering the ledger (see analysis of Lemma 9), and settle. This is guaranteed to happen within another $\eta + u$ time slots. $\square$

Theorem 4. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Full Leios achieves $(\varepsilon_{\text{tp}}(t), R)$ -throughput for $\varepsilon_{\text{tp}}(t) = \varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$ and $R = \left(\frac{1-\varepsilon}{1+\varepsilon} \frac{\lambda-\rho}{\lambda+1}\right) (1 - \varphi_{\text{fail}})\alpha_H C$ with $\varphi_{\text{fail}} = \mathcal{O}(\lambda^2)\varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L)$.*

Proof. Along the lines of the proof of Theorem 2, this theorem follows from the fact that all successful pipelines are covered in the mainchain, and from the maximal rate φ_{fail} of unsuccessful pipelines as observed in Lemma 8.

In particular, to satisfy the condition of Definition 4, for $\gamma > 0$, t_0 can be chosen such that

$$(1 - \gamma) \geq \frac{t_0 - t_P L - u - \eta}{t_0} \quad (2)$$

as in the proof of Theorem 2.

Finally, note that for any security parameter κ , $\varepsilon_{\text{tp}}(t) = \varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$ remains negligible over execution times polynomial in κ (by a simple union bound), and thus the throughput condition holds except for negligible error in κ . $\square$

Theorem 5. *Let ε be such that $0 < \varepsilon < 2\alpha_H - 1$. Full Leios achieves $(\varepsilon_{\text{tp}}(t), R)$ -throughput for $\varepsilon_{\text{tp}}(t) = \varepsilon_{\text{base}}(t) + \varepsilon_{\text{vote}}(t)$ and $R = \left(\frac{1-\varepsilon}{1+\varepsilon} \frac{\lambda-\rho}{\lambda+1}\right) \alpha_H C$ which is near-optimal.*

Proof. By Theorem 4, for any pair (γ, t_0) satisfying the condition of Definition 4 we have

$$\Pr \left[|L_{\text{san}}^{(t)}|/t < (1 - \gamma)(1 - \varphi_{\text{fail}})R \right] \leq \varepsilon_{\text{tp}}(t)$$

and thus

$$\Pr \left[|L_{\text{san}}^{(t)}|/t < (1 - \gamma')R \right] \leq \varepsilon_{\text{tp}}(t)$$

for $0 < \gamma' < \gamma - \varphi_{\text{fail}}$ with some sufficiently small $\varphi_{\text{fail}} > 0$.

We now give an estimate on the order of t_0 . From Lemma 5, we recall that

$$\varepsilon_{\text{fail}} := \varepsilon_{\text{fail}}(\varepsilon, f_I, f_E, L) := e^{-\varepsilon^2 \Omega(f_I L)} + e^{-\varepsilon^2 \Omega(f_E L)}$$

and that

$$\varphi_{\text{fail}} = \mathcal{O}(\lambda^2)\varepsilon_{\text{fail}}$$

and that Condition (2) (with respect to our new γ') is satisfied for

$$t_0 = \mathcal{O}(t_P L / \gamma') = \mathcal{O}(\lambda^3 L / \gamma').$$

We get

$$t_0 = \mathcal{O}(\lambda^3 L / (\gamma - \mathcal{O}(\lambda^2 \varepsilon_{\text{fail}}))),$$

where $\varepsilon_{\text{fail}}$ vanishes exponentially in L .

Finally, the throughput is near-optimal as it can be brought arbitrarily close to $\alpha_H C$, were, in any protocol, the honest parties cannot include more than their fair share of payloads which is limited at rate $\alpha_H C$. $\square$

H Protocol Extensions

H.1 Multi-Epoch Protocol

To lift the overlay protocol to span multiple epochs (reflecting underlying stake change over the course of time), the respective mechanisms of the base protocol can be used (as for instance explicitly given in [21]). Once the underlying base protocol outputs the new stake distribution, that one is simply used for the IB/EB lottery and voting.

H.2 Different Consensus Resources

Our protocol extension is applicable to protocols based on other resources than stake, e.g., proof of work (PoW). Note, however, that the block/vote lotteries are tightly coupled to the pipeline stages—enabled by the fact that the VRF lotteries are tied to a particular time slot. To match these guarantees in a PoW protocol, this would make it necessary for the PoW to additionally prove that it was not computed ahead of time. Such proofs can be given, e.g., by referencing a stable block in the base protocol (a mechanism used in [51])—but still giving the adversary a high degree of freedom of what “time stamps” to effectively choose (e.g., by referencing a newer, unstable block). To compensate this effect, the pipeline length L would have to grow to levels that would push liveness into intolerable regions.

This problem can be avoided by applying a mechanism in the spirit of [28]: input blocks, additionally to carrying transactions, also serve to measure each party’s work contribution during the epoch—measured by their ratio of input blocks contributed. This measure is then transformed into a “virtual” stake distribution among the parties at the end of the epoch. Any PoS base protocol can then be applied while substituting stake by virtual stake.

Note that, as a consequence, the security of the underlying PoW protocol is somewhat degraded as public-key cryptography needs to be introduced, opening it up for new attack vectors (long-range attacks or the disability to recover from adversarial spikes).

H.3 Robustness under Extreme Conditions

Up to now we have analyzed the behavior of Leios under favorable network and adversarial conditions, i.e., the network does not experience any outages and honest parties control a majority of the stake. However, from a robustness perspective it is critical that the protocol remains secure under temporary disruptions such as network outages, even if these conditions are only occurring rarely. Otherwise, we risk the whole protocol collapsing from a single “black swan” event.

We examine here two such “bad” events of interest: temporary network outages, and temporary control of the relative (but not absolute) majority of the stake by the adversary. Given that for consensus Leios is dependent on the base-protocol, we cannot hope to get better guarantees than those provided by it. Hence, the main question which we explore in this section, is what security and performance characteristics Leios preserves when such events occur, assuming the base-protocol can tolerate them. Concretely, we are going to assume that (i) there is always enough bandwidth to safely run the base-protocol, and (ii) that the base-protocol self-heals, i.e., if agreement is disrupted due to a temporary dishonest relative majority, the system recovers after some time with parties again reaching a state of agreement.²⁴

Assume now that both “bad” events occur. First off, given that no *absolute* stake majority is formed, the worst thing that can happen in the Leios layer is either no EB being certified or an EB containing only adversarial IBs being certified. In any case, no purely adversarial and possibly incorrect EB certificates are produced, and thus *IB availability* is always ensured albeit with some extra delays in delivery due to the temporary network outages. Now, our effort earlier in decoupling consensus from block availability pays out as only IBs for which delivery is guaranteed are serialized by the base-protocol. Hence, IB delivery or EB certification failures do not affect consistency. Similarly, in the case where the consistency of the base-protocol is temporarily disrupted, the recovery of the base-protocol directly implies the recovery of Leios. Summarizing, Leios inherits the self-healing property of the base-protocol, while throughput and liveness may degrade temporarily when such disruptions occur.

Finally, due to the modular design of Leios, and the fact that only a single EB certificate needs to be included in each base-protocol block, throughput and liveness at a similar level as that of the base-protocol can be guaranteed under such extreme conditions. Specifically, the base-protocol can serve a *dual* role by containing both an EB certificate and regular transactions. Now, even if no honest IB makes it to a certified EB or no certified EB is produced at all, some of the throughput is still retained intact due to the regular transactions being included in the base-protocol blocks.

²⁴In fact, both PoW and PoS low-throughput blockchain protocols have been shown to provably satisfy this property [2,4], starting with Bitcoin.